

Extracting large-scale neural activity with Suite2p

Carsen Stringer^{1†}, Chris Ki², Nicholas DelGrosso³, Paul LaFosse¹, Qingqing Zhang¹, Marius Pachitariu^{1†}

¹HHMI Janelia Research Campus, Ashburn, VA, USA

²Carnegie Mellon University, Neuroscience Institute and Dept of Machine Learning, Pittsburgh, PA, USA

³Universität Bonn, Institute for Experimental Epileptology and Cognitive Sciences, Bonn, Germany

† correspondence to ([stringerc](mailto:stringerc@janelia.hhmi.org), [pachitariu](mailto:pachitariu@janelia.hhmi.org)) @ janelia.hhmi.org

Abstract: Neural recordings using optical methods have improved dramatically. For example, we demonstrate here recordings of over 100,000 neurons from the mouse cortex obtained with a standard commercial microscope. To process such large datasets, we developed Suite2p, a collection of efficient algorithms for motion correction, cell detection, activity extraction and quality control. We also developed new approaches to benchmark performance on these tasks. Our GPU-accelerated non-rigid motion correction substantially outperforms alternative methods, while running over five times faster. For cell detection, Suite2p outperforms the CNMF algorithm in Caiman and Fiola, finding more cells and producing fewer false positives, while running in a fraction of the time. We also introduce quality control steps for users to evaluate performance on their own data, while offering alternative algorithms for specialized types of recordings such as those from one-photon and voltage imaging.

Introduction

Optical recording methods are useful in a wide variety of neuroscience experiments [1, 2]. They allow for easy multiplexing of an activity indicator with other optical markers, for example to determine cell types or projection targets [3, 4]; they give exact tissue position for each recorded neuron, and enable sampling of wide areas to look for changes in tuning within and across brain areas [5, 6]; they allow for easy demixing of activity between nearby neurons simply in virtue of their spatial separation; they enable tracking of the activity of large neural populations across weeks and months [7, 8]; they also allow for scaling up of neuronal yields at the expense of temporal resolution [9, 10]; miniaturized microscopes enable imaging of the same population over many hours [11, 12] etc. However, optical recordings typically produce large amounts of raw data, which need to be analyzed and processed in order to obtain interpretable neural activity timecourses. Suite2p is a collection of algorithms developed specifically for this purpose.

In the analysis of optical recordings, two processing steps have attracted the most research attention, namely the spike detection step and the cell detection step. For spike detection, we do not introduce a new algorithm here and instead rely on a fast implementation of OASIS [13] (see Discussion). Cell detection is the task of creating pixel masks that cover the somas or other biological compartments of a cell. This is typically done using functional data, because pixels in different cells are high at different times, while pixels in the same cell are active simultaneously [14–27]. The task of cell detection is well-matched to matrix

decomposition problems, for which a large number of approaches are readily available. The Suite2p detection algorithm can also be interpreted as a matrix decomposition approach, but it relies on strong L0 sparsity constraints on both matrix factors, for which standard solutions do not exist.

Less research attention has been directed towards the registration and quality control steps. While most types of registration rely on frame correlation approaches, the implementation and accuracy of these methods can vary substantially, as we shall see below. Recent methods have claimed improvements in processing speed from running on GPUs, though we have found these claims to only be valid at very small frame sizes (<200 pixels). Finally, some type of quality control step would be useful after each data processing step, but there has been limited development in this area. For example, quality control steps can show a user whether the registration was successful, whether Z drift was encountered in the recording, whether cells have been extracted uniformly from across the entire field of view, whether neural activity patterns are dominated by artifacts, etc.

Results

The development of Suite2p has continued uninterrupted over the past 10 years, starting with the algorithms in the original preprint [29]. Here we describe and evaluate the algorithms in Suite2p as they exist today, serving as a reference at this particular point in time. The current version (Figure 1a-e) includes: 1) support for multiple input file types; 2) GPU-accelerated motion registration in 2D and across Z-stacks; 3) multiple algorithms for cell

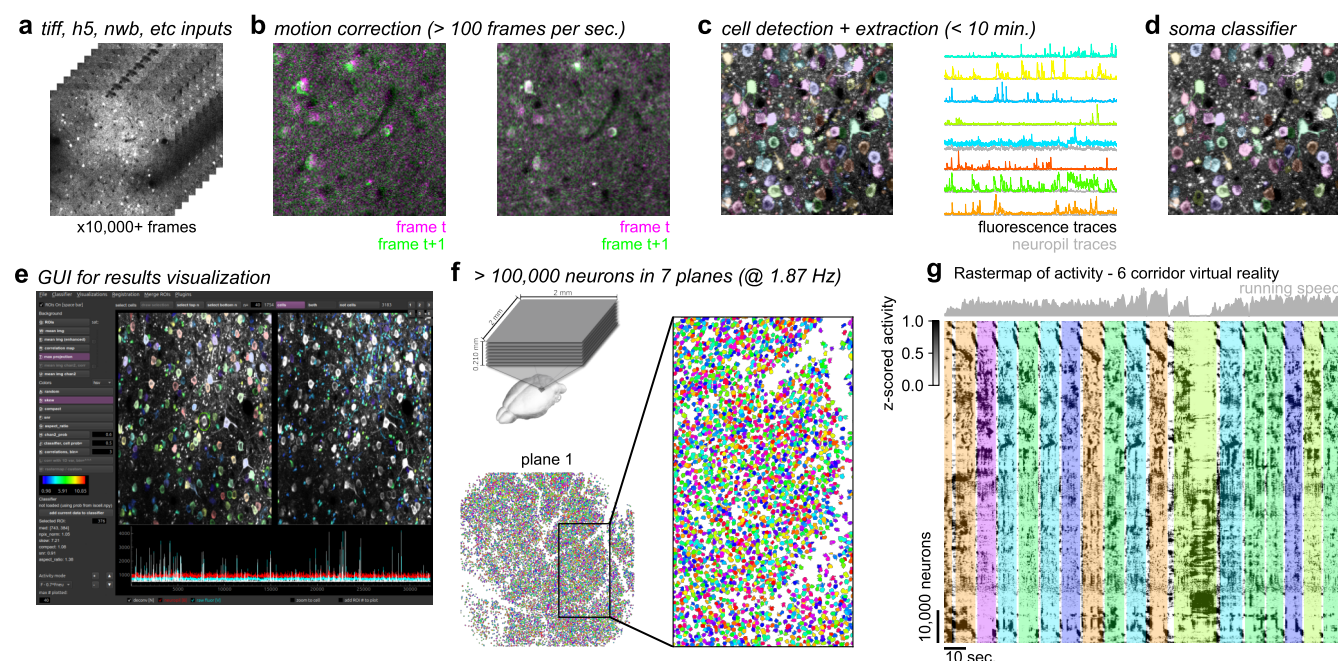


Figure 1: Suite2p pipeline. **a**, Example volume imaging data. **b**, X/Y motion correction (left: before, right: after). **c**, ROI detection (left) and trace extraction with neuropil extraction based on surrounding pixels (right). **d**, Classification of ROIs into somatic and non-somatic (i.e. dendrites and other neuropil). **e**, The graphical user interface (GUI) can be used for manual curation and results visualization. **f**, Multi-plane recording of over 100,000 neurons using a standard Thorlabs Bergamo2 scope (see Methods for exact configuration). Somatic ROIs for example plane shown in different colors. **g**, Neural activity raster from recording in **f**, with neurons sorted by Rastermap, colored by the corridor that the mouse is in [28].

segmentation relying on either functional or anatomical information; 4) GPU-accelerated activity extraction with neuropil correction and spike deconvolution; and 5) ROI classification and quality control, integrated in a graphical user interface (GUI) with extensive functionality (Figure 1e, Figure S1). Suite2p completes all tasks in a fraction of the acquisition time on a standard desktop computer with a consumer-grade GPU. The software package is available at github.com/MouseLand/suite2p.

We demonstrated in the original Suite2p preprint that large-scale recordings of over 10,000 neurons are possible on commercially-available microscopes such as the Thorlabs Bergamo2, which served as motivation for the development of highly-efficient algorithms that are also highly-accurate so that manual curation was no longer necessary. Here we show that on the same type of microscope, recording size can be pushed above 100,000 neurons simultaneously, further emphasizing the requirements of high efficiency and high accuracy for data analysis (Figure 1fg). The increase was made possible by the following technological upgrades: 1) a fiber laser with lower repetition rate laser (30Mhz) and short pulse widths (100fs) with GDD compensation; 2) an objective with lower zoom and NA (Nikon 10x, 0.5 NA) while

maintaining high collection efficiency; 3) new riboL1-based jGCaMP8s transgenic mouse line [30]. Note that the microscope itself did not need to change.

Motion correction

Brain motion on the order of several microns is very common in awake, in vivo recordings and thus it must be corrected computationally, ideally with sub-micron precision. Since the two-photon, pixel-by-pixel acquisition is sequential, brain position can change over the course of a single frame, resulting in a non-rigid deformation of the final frame [31]. Non-rigid deformations can also be induced by other factors, generally on longer timescales, such as very slight angle changes, laser beam walk and blood flow changes. These non-rigid deformations have to be addressed computationally in order to maintain sub-micron registration accuracy across the entire field of view.

Across biological applications, a large array of registration methods exist. To choose between them, we consider a number of factors unique to neural recordings: 1) low SNR / high shot noise on a frame-by-frame basis; 2) large number of frames per recording (hundreds of thousands); 3) non-rigid, but locally-smooth deformations. The first factor implies that we

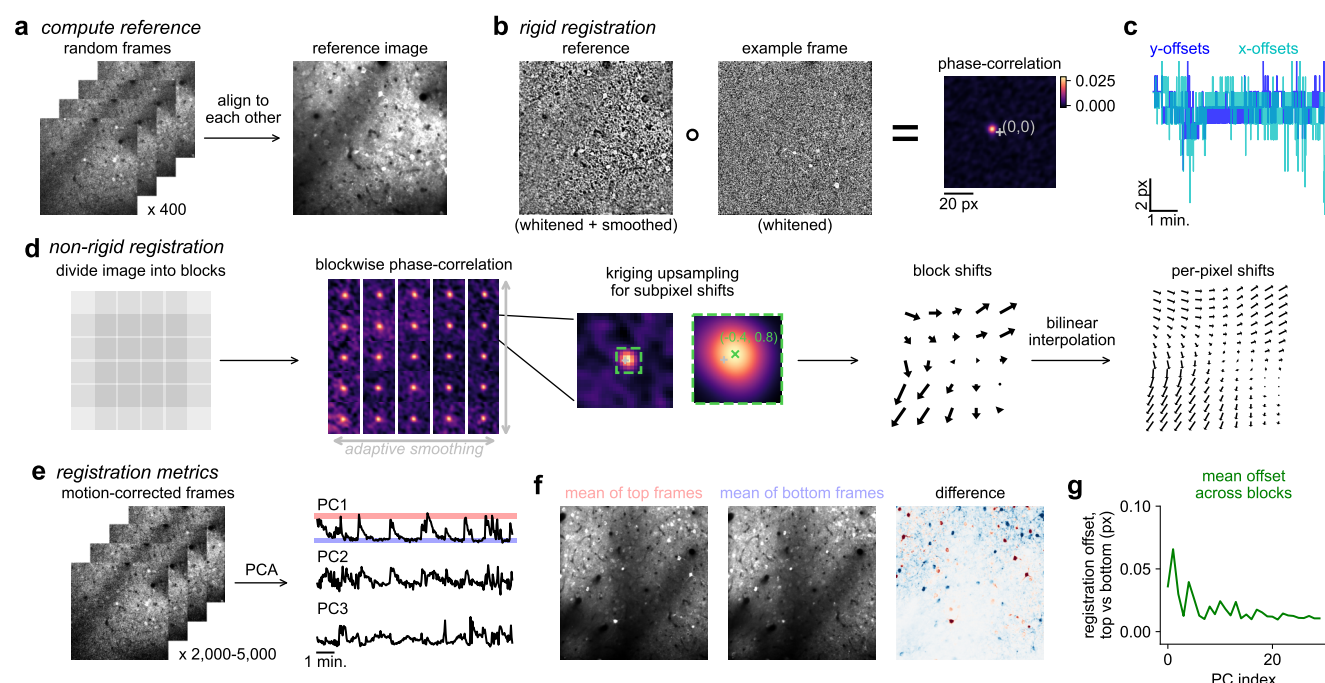


Figure 2: Motion correction and stability metrics. **a**, Reference image is computed by iterative re-alignment of a random subset of frames. **b**, Rigid motion offsets are computed by correlation of whitened reference and frame, i.e. phase correlation. **c**, Calculated X/Y rigid motion offsets. **d**, Non-rigid motion offsets computed by dividing image into overlapping blocks, computing phase correlation in each block, smoothing across blocks, taking the optimal shifts per block and upsampling them via bilinear interpolation. **e**, Registration performance is assessed by applied principal components analysis (PCA) to the motion-corrected frames. The PCs are expected to correspond to neural activity if all motion is corrected. **f**, Mean of the top PC1 frames (left), bottom PC1 frames (middle) and their difference (right). **g**, Registration offset lengths between top and bottom frame means.

should take advantage of global motion estimation as much as possible, to integrate over the SNR of the entire frame. The second factor implies that processing should be fast, and the final factor implies that some amount of non-rigid correction must be done, even though locally there might not be sufficient information to estimate the motion due to the first factor.

To begin motion correction, it is necessary to create a reference image. The quality of the final registration depends critically on the resolution and SNR of this image. Due to low SNR per frame, a single frame cannot be used as reference. While an average of the first few hundred frames can be used [32–34], this average generally produces a blurry image due to the existence of brain motion (Figure S3a). To fix this problem, we refine the reference image iteratively from a subset of frames, averaging only frames that are well-aligned to the target at every iteration (Figure S3b, Figure 2a).

The second step computes a global correction for each frame, which we do using a rigid phase-correlation approach. Frame correlation approaches in general have the advantage they can be efficiently computed in the Fourier domain, and they integrate information across the frame to produce a single

global registration offset. While previous methods like NoRMCorre and Caiman use a “cross”-correlation approach [33, 35], we found “phase”-correlation to perform substantially better ([36, 37], Figure 2b). The difference between these two is a whitening of the frequency spectrum in phase correlation, which emphasizes edge-like features in the image, rather than the low frequencies which are typically less informative about precise position.

After global, rigid correction per frame, the third step applies a non-rigid correction. This step compensates for the differential movements that different portions of the frame may contain, and thus is a relatively local, smaller correction. We determine this correction by dividing the frame into blocks, similar to other methods [35], and computing the phase correlation for each block. At this stage, we could directly correct each block, similar to previous methods [35]. However, such a correction may be quite noisy, because individual blocks contain a fraction of the global information available in the image, and optical recordings are generally made with large shot noise. Thus, we apply an adaptive smoothing step, which smooths the phase correlation maps across adjacent blocks, with a smoothing constant that depends on the estimated

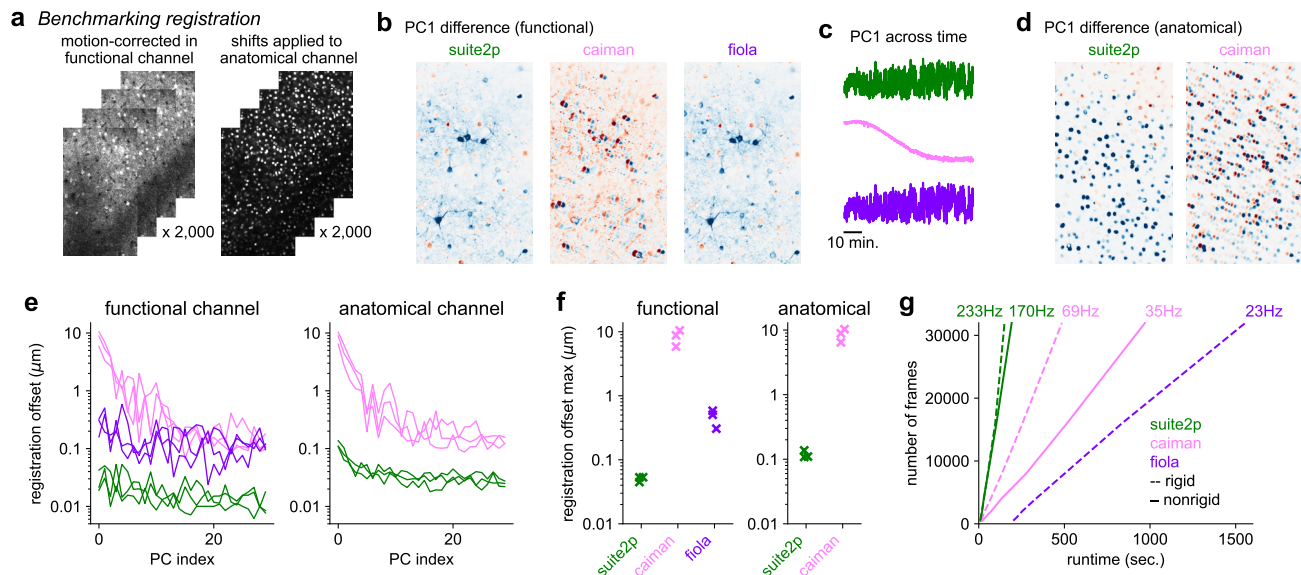


Figure 3: Benchmarking motion correction in vivo. **a**, Recordings of neurons expressing both GCaMP6s and tdTomato. Motion correction shifts were computed using the functional channel, and applied to the anatomical channel. **b**, Difference between top frames mean and bottom frames mean for PC1 (as in Figure 2f) for motion correction performed using Suite2p, Caiman, and Fiola, for the functional channel. **c**, Time course of PC1 across frames. **d**, Same as **b** on the anatomical channel. **e**, Registration offset lengths (like Figure 2g) for the functional (left) and anatomical channels (right). **f**, Maximum registration offset across the top 30 PCs for the functional (left) and anatomical (right) channel. **g**, Runtime performance. Frames per second reported as slope of linear fit.

SNR for each block. Thus, blocks with high SNR will have an accurate and very local phase correlation map, while blocks with low SNR will have a more global map, but still be relatively accurate. Spatial offsets for each block are estimated from the peaks of the smoothed phase correlation maps after it has been spatially upsampled by a factor of 10 using Kriging interpolation with a 2D Gaussian kernel [38]. This upsampling enables subpixel corrections. The spatial offsets at each block are then interpolated to each pixel in the image, and applied to the frame using bilinear interpolation to generate the final corrected image.

Registration benchmarks and quality control

To determine the success of motion correction, we designed a set of registration metrics that can be applied to any recording and any algorithm. Assuming all motion in the recording has been removed, the top principal components (PCs) of the corrected movie should be related to the activity of the somas or other compartments. If residual motion is left, it will correlate pixels over large spatial extents, and thus the motion will manifest in the top PCs (Figure 2e). We thus investigate the top PCs individually, by averaging the “top” frames with the largest projections onto the PC, and the “bottom” frames with the lowest projections (Figure 2f), and register these averages to each other. The amplitude of the offsets can provide an estimate of residual motion related to this PC (Figure 2g). We

can also determine whether the residual motion is rigid or non-rigid, simply by estimating offsets with rigid/non-rigid registration. The difference between top and bottom frames can be inspected for qualitative assessment, and may provide insight into the type of residual motion present [39]. We make these metrics and corresponding visualizations easily available in the Suite2p GUI (Figure S1).

Beyond helping users understand the motion in their recordings, the registration metrics also enable a straightforward comparison between different algorithms on real data. We benchmarked Suite2p against Caiman [33], a popular calcium imaging processing tool, and Fiola [34], a more recent processing tool with GPU-accelerated registration. We ran Suite2p, Caiman and Fiola on a multi-ROI recording with both a functional and an anatomical channel (Figure 3a). We used the functional channel to determine motion offsets that were applied to both the functional and anatomical data and then computed the registration metrics (Figure 3b-d). Using the non-rigid registration offsets, we found that Suite2p outperformed Caiman and Fiola on the functional and anatomical channel metrics (Figure 3ef, Fiola cannot apply the anatomical channel correction). The metrics also indicated a gradual temporal misalignment for Caiman, which we verified by inspecting the offsets (Figure S4) and which we attribute to the incremental, “online” updating of the template image.

The Suite2p rigid and non-rigid registration algorithms were substantially faster than Fiola and Caiman, even when Caiman was run on a recent compute cluster node with 16 cores, and with Fiola running on the same GPUs as Suite2p (Figure 3g). We note that Fiola does not support non-rigid registration, and the speedups previously reported in relation to Caiman may have been due to the specific configuration and hardware used there [34].

ROI detection

Once the movie is registered, the next step is ROI detection. Typical matrix decomposition approaches tend to extract many ROIs that are not single cellular compartments, such as neuropil and motion artifacts, because these sources induce correlated factors across pixels. While such factors can be classified post-hoc and eliminated, they can affect the detection and activity extraction of overlapping cellular compartments. In our experience, we find such artifactual ROIs to be easily identified by their lack of activity transients. Conversely, if an ROI has even a single high-activity transient, it likely corresponds to a cellular compartment. This led us to develop an algorithm that would specifically detect ROIs by their transient excursions above baseline. The algorithm can be formulated as matrix decomposition with dual L0 penalties on spatial and temporal factors (see Methods). However, it can also be described in more intuitive terms, which we do below.

To start, the registered movie is preprocessed by binning and high-pass filtering (Figure 4a). We then convolve each binned frame with square templates of different sizes and threshold at a fixed high value to identify transients, which are then summed up to generate a score, or “variance explained” for each template size (Figure 4b). This score corresponds to the data variance accounted for by the model if we were to introduce a square template of that size at that particular location. From here, the algorithm proceeds iteratively, identifying the highest peak across variance explained maps, and optimizing a template centered on that peak in order to explain more variance (Figure 4c-e) which leads to a refinement of its shape. The activity of the ROI is then subtracted, the variance explained maps get updated efficiently, and the next ROI is extracted in a similar fashion, until no ROIs are left above a certain variance explained threshold, similar to matching pursuit algorithms [41].

This algorithm allows for overlapping ROIs to be found (Figure 4f). Due to its multi-scale nature, it can detect various types of compartments in the field of view, including somas and dendrites (Figure 4g).

To evaluate the algorithm, we generated hybrid

ground-truth recordings based on real in vivo recordings. Our methodology relies on a riboL1-based calcium indicator which localizes to the soma, and thus makes cellular identification straightforward, even using anatomical methods like Cellpose (Figure 5a) [40, 42]. Similarly, traces were extracted just by averaging pixels. To make the benchmark challenging, we added simulated neuropil, both as individual localized compartments (dendrites/axons) and as a spatially-diffuse, averaged signal (Figure 5a). The timecourses of these added components were obtained from real activity traces recorded in the other imaging planes. Finally, while the original recording was acquired at high laser powers, we added shot noise to make the hybrid ground-truth more representative of standard recordings.

We compared Suite2p to the CNMF-based [18] algorithm from Caiman, which was kept unmodified in Fiola for ROI detection (Figure 5b). Across recordings, Suite2p maintained a higher F1 score, with fewer false negatives and fewer false positives (Figure 5c). The higher F1 score of Suite2p was maintained when varying the parameters of the simulation (Figure 5d, Figure S7). Suite2p also ran faster than Caiman, averaging 125 seconds for detection and extraction on the simulations, compared to 1104 seconds for Caiman. We note that we optimized a single parameter for both Suite2p and Caiman to finetune the total number of extracted ROIs, which had a small effect on performance (Figure S5). We also optimized together five Caiman parameters, which led only to very modest increases in performance (Figure S6), so we did not include this optimization in the main results to prevent overfitting.

Within Suite2p, we also include two other detection algorithms, Sourcery and Cellpose, which are useful in low-SNR scenarios and for different sensors (see Methods).

Extraction, deconvolution and GUI

The final steps in the pipeline are activity extraction and spike deconvolution. For the former, we extract traces simply as averages of pixels, taking care to remove portions of each ROI that overlap with others as a conservative approach. Other methods like Caiman and Fiola use overlapping pixels and use source deconvolution to separate signals, which may be biased by constraints and noise levels. Similarly for spike deconvolution we adopt a simple algorithm, OASIS with the AR-1 model and no sparsity constraints [13, 43]. Finally, the Suite2p GUI was designed to efficiently display multiple image projections, ROI metrics, correlations between ROIs, and population activity visualization via Rastermap [28] (Figure S1).

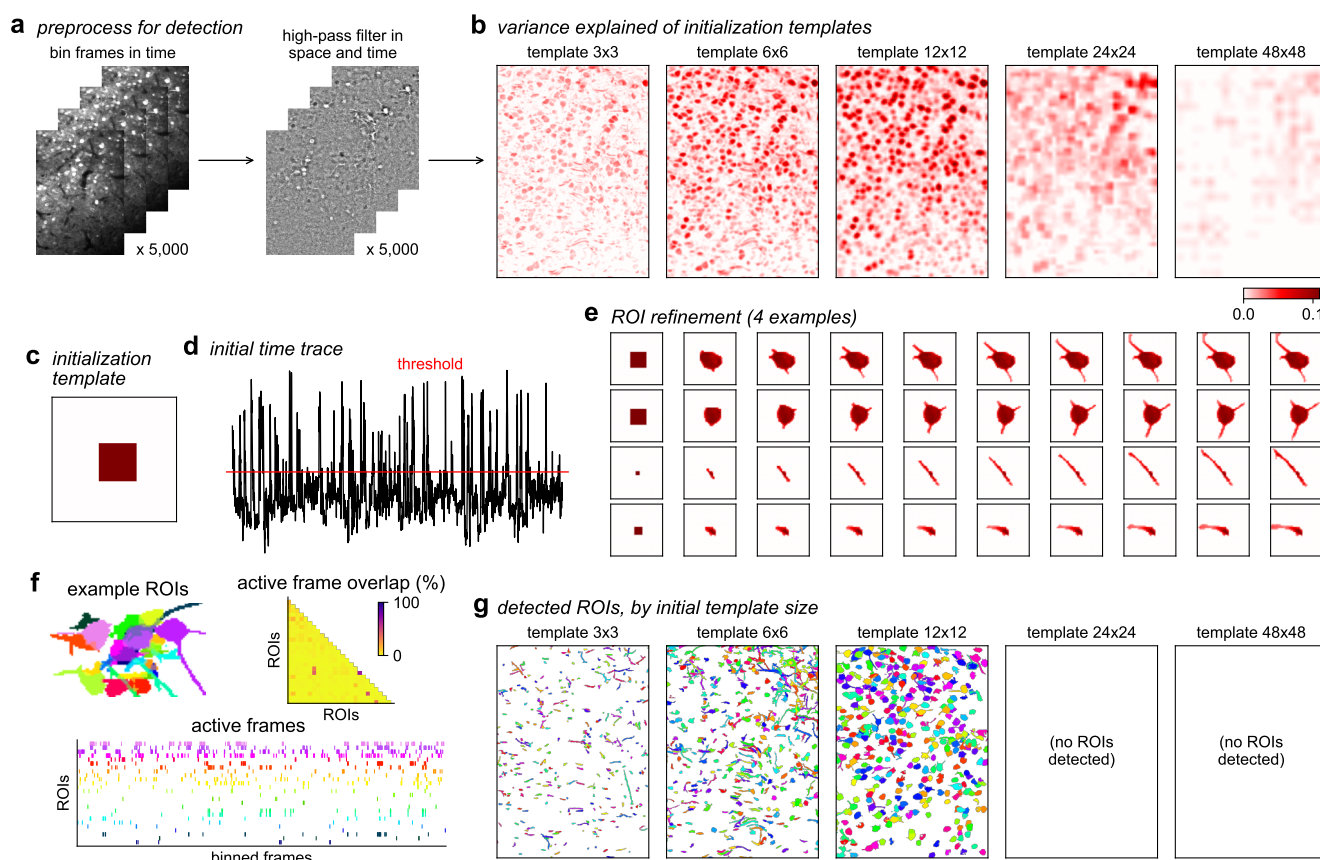


Figure 4: ROI detection. **a**, Frames are binned in time (left) and then high-pass filtered in space and time (right). **b**, Time-averaged thresholded activity from convolved square templates of different sizes. **c**, Initialization of new ROI with best fitting square template from **b**. **d**, Initial time trace corresponding to the dot product of the template in **c** with each movie frame. **e**, Iterative template refinement for four example ROIs. **f**, Example region with many cells detected and their spatial overlaps (top left), temporal overlaps (top right) and active frames crossing the threshold (bottom). **g**, Detected ROIs divided by the sizes of their initial square templates.

The GUI can also be used to manually classify ROIs, but we recommend using this functionality sparingly, and focusing on changing algorithm parameters or recording configurations to obtain better automated results. In our experience, checking each recording in Suite2p immediately after acquisition is an invaluable step that allows for early detection and debugging of problems with the recording or the data processing.

Discussion

Suite2p provides a complete suite of algorithms for extracting neural activity from optical recordings. As a practicing lab, we perform several large-scale neural recordings every day, which has allowed us to thoroughly test the robustness of the pipeline. Using the Janelia GPU cluster and parallelizing the data processing over planes, we now obtain high-quality processed datasets within 20 minutes of a recording finishing. This fast feedback allows for quick experimental iterations and faster debugging and

improvement of recording configurations.

The fast registration algorithms in Suite2p are also integrated with Scanimage [44] to provide realtime Z-drift estimation and correction. We have found this step to be crucial in maintaining stability over long recording durations. Suite2p also aligns dense z-stacks if volumetric data is provided, and the results can be visualized in the GUI.

For spike deconvolution, multiple types of algorithms exist [13, 45–52], with newer ones based on deep neural networks [53–57]. All of these algorithms take as input extracted calcium traces, and thus can be incorporated easily at the end of any pipeline. For Suite2p, we include an established approach of OASIS deconvolution [13], which runs very efficiently on CPUs and assumes a minimal model of fluorescence kinetics. We use the simple AR-1 model from OASIS, which requires the least compute resources. Newer sensors like jRCaMP8 [58] have conveniently pushed the fluorescence kernel closer to the AR-1 model

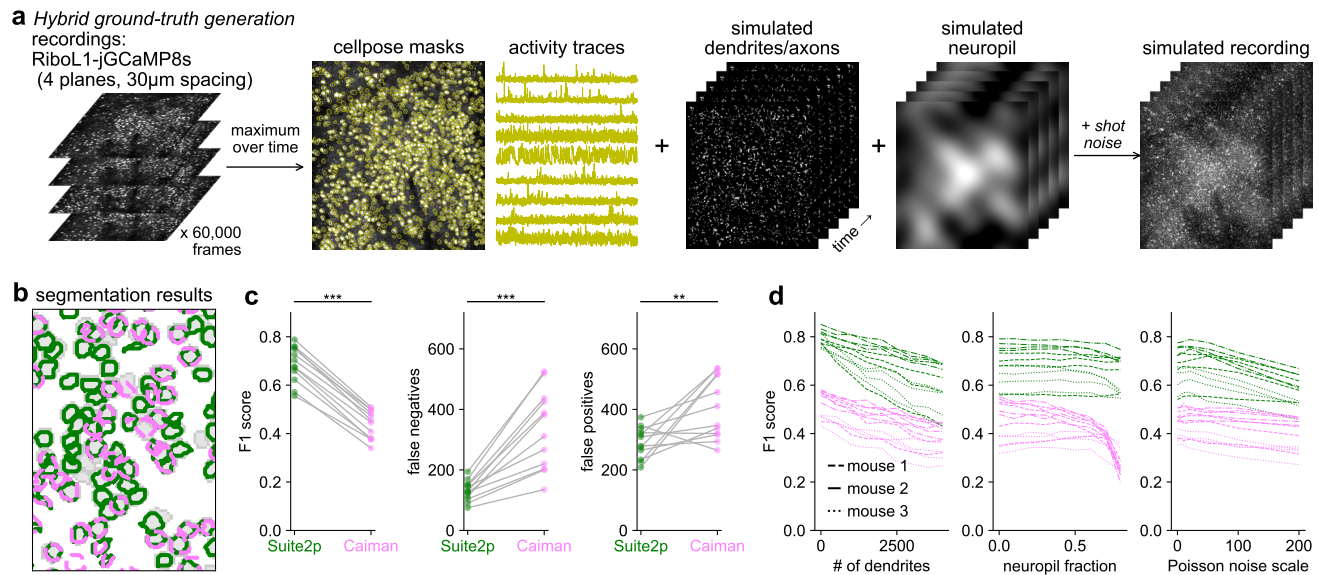


Figure 5: ROI detection benchmarks with hybrid ground-truth. **a**, Hybrid ground-truth generation using recordings of visual cortical neurons expressing riboL1-jGCaMP8s [40]. Anatomically detected ROIs were used as ground truth. We added simulated processes and neuropil using activity from other recorded planes. **b**, Segmentation results from Suite2p (green) and Caiman (pink) overlaid on ground-truth ROIs (gray). **c**, F1 scores (left), false negatives (middle), and false positives (right) relative to the anatomical ground truth (paired two-sided t-test, $n=12$ simulations). **d**, F1 scores across sweeps of simulation parameters. The center values of 2,000, 0.4 and 20 respectively were used in **bc**.

assumptions of OASIS, with near-zero rise times and short, exponential decay that suffers less from the nonlinear effects of jGCaMP7 and earlier versions [59, 60].

Acknowledgments

This research was funded by the Howard Hughes Medical Institute at the Janelia Research Campus. From the Vivarium, we thank Jim Cox, Crystall Lopez, Anne Kuzspit, Miriam Rose, Alexa Gracias, Gillian Harris, Sarah Lindo, and their respective teams for animal breeding, husbandry, and surgeries. From JeT, we thank Daniel Flickinger, Vasily Goncharov, Alex Sohn, Tobias Goulet, and Steven Sawtelle for help with rig maintenance and upgrades. We thank Sverre Grødem, Kristian K Lensjø, and Marianne Fyhn for the RiboL1-jGCaMP8s virus. From MBF Bioscience we thank Georg Jandl, Mitchell Sandoe, and Boris Djiguemde for scanimage support. We thank members of our lab for testing the latest Suite2p version. We thank Kenneth Harris and Matteo Carandini for supporting the development of an earlier version of Suite2p, and Sylvia Schroeder and Mario Dipoppa for testing and contributions to an earlier version. For testing an earlier version, we also thank Federico Rossi, Ross Williamson, Dimitar Kostadinov, Henry Dalgleish, Michael Hausser, Alice O'Leary and Asaph Zylbertal. We thank Minh Thao Nguyen, Andrea Pierré, Joao Couto, and Tolga Dincer for code contributions

to improve file format support. We thank Jan Willem de Gee for contributing an option for running motion correction multiple times, and Seetha Krishnan for contributing a manual ROI drawing tool.

Data availability

All datasets will be available upon publication of the paper in a journal.

Code availability

The code and GUI for Suite2p are available at <https://www.github.com/mouseland/suite2p>. Scripts for recreating the analyses in the figures are available at <https://github.com/MouseLand/suite2p/tree/main/paper/>.

Methods

The Suite2p code library is implemented in Python 3 [61], using numpy, scipy, pytorch, numba, scikit-learn, tifffile, and ScanImageTiffReader for processing [62–68], and pyqt, pyqtgraph, and superqt for the GUI [69–71]. The figures were made using matplotlib and jupyter-notebook [72, 73]. All experimental procedures were conducted according to IACUC, ethics approval received from the IACUC board at HHMI Janelia Research Campus.

Data acquisition

Animals

All experimental procedures were conducted according to IACUC. We performed recordings in one C57Bl/6J mice, three VGAT-CRE x Ai14 mice (JAX 016962, JAX 007914), and one mouse bred to express jGCaMP8s in excitatory neurons: TIGRE2-RiboL1-jGCaMP8s-IRES-tTA2 (variant of JAX039267 without IRES and with WPRE) x Slc17a7-cre (JAX 023527) [30]. These mice were male and female, and ranged from 2 to 12 months of age. Mice were housed in reverse light cycle, and were housed with siblings before and after surgery. Holding rooms are set to a temperature of 70°F +/- 2°F, and humidity of 50%RH +/- 20%.

Surgical procedures

Surgeries were performed in adult mice (P56–P200) following procedures outlined in [74]. In brief, mice were anesthetized with Isoflurane while a craniotomy was performed. Marcaine (no more than 8 mg/kg) was injected subcutaneously beneath the incision area, and warmed fluids + 5% dextrose and Buprenorphine 0.1 mg/kg (systemic analgesic) were administered subcutaneously along with Dexamethasone 2 mg/kg via intramuscular route. Measurements were taken to determine bregma-lambda distance and location of a 4 mm circular window over visual cortex, as far lateral and caudal as possible without compromising the stability of the implant. In VGAT-CRE mice, injections were performed of soma-targeted RiboL1-jGCaMP8s (a mixture of AAV9-Thy1s:TTA and AAV9-TRE3G:RiboL1-jGCaMP8s) [40, 75]. In the C57 mouse, injections were performed of AAV-hSyn1-GCaMP6s-P2A-nls-dTomato, (Addgene plasmid # 51084 ; <http://n2t.net/addgene:51084> ; RRID:Addgene_51084). A 4+5 mm double window was placed into the craniotomy so that the 4mm window replaced the previously removed bone piece and the 5mm window lay over the edge of the bone. After surgery, Ketoprofen 5mg/kg was administered subcutaneously and the animals allowed to recover on heat. The mice were monitored for pain or distress and Ketoprofen 5mg/kg was administered for 2 additional days following surgery.

Imaging acquisition

During the recordings, mice were free to run on an air-floating ball or a treadmill. Mice were acclimatized to running head-fixed for several sessions before imaging.

We used ScanImage [44] for data acquisition, and a custom online Z-correction module for the recording in Figure 1, to correct for Z and XY drift online during the

recording, called the “MariusMotionEstimator” and the “MariusMotionCorrector” in ScanImage.

For the many plane recording (Figure 1), we used a commercial two-photon ThorLabs Bergamo2 microscope with a piezo for multiplane acquisition, an acquisition system from MBF Bioscience (the “slow” vDAQ), and an objective with lower zoom and NA (Nikon CFI Plan Apo 10XC Glyc, 0.5 NA, WD 5.5mm, MRD71120). We used a Toptica Femtofiber 920nm laser customized to a repetition rate of 30Mhz with a max power at the sample of approximately 240mW. We recorded eight planes one of which was a flyback plane, at a rate of 1.87Hz, with pixel sizes of 2um in both X and Y (Figure S2). The laser pulses were synchronized with scanimage, allowing for precise matching of pulses to pixels. While possible on this setup, we did not use the uniform sampling mode. During the experiment, the mouse ran in a closed-loop virtual reality with eight different corridors.

For the benchmarking experiments, we used a custom-built 2-photon mesoscope [76] to record neural activity. For the registration benchmarking, the XY drift correction was turned off. For the detection benchmarking experiments, we used an upgrade of the mesoscope that allowed us to approximately double the number of recorded neurons using temporal multiplexing [77], as described in [74].

Overall pipeline settings

There are several settings, saved in ‘settings.npy’, that the user is recommended to modify for their experiment:

- `torch_device`: Torch device for performing operations, GPU is ‘cuda’ and CPU is ‘cpu’
- `tau`: Timescale of decay of indicator in seconds.
- `fs`: Sampling frequency of each plane in Hz, i.e. the volume rate of the acquisition.
- `diameter`: Expected ROI diameter, used for detection with Cellpose or original Suite2p algorithm, Sourcery.

Input formats and settings

Suite2p supports tiffs, hdf5 files, NWB files, bruker tiffs, avi files, and dcim files. Metadata about the recording and file saving locations is provided in a dictionary ‘db.npy’, or through the GUI, which is then converted to an npy file. These settings are used to convert the inputs into binary files which are memmapped during the processing. Each ScanImage ROI and imaging plane is saved as a separate binary file in a separate folder ‘planeX’, where X is a number iterated over ROIs and planes. The input settings include

- `data_path`: List of folders with tiffs or other files to
- `input_format`: Can be ['tif', 'h5', 'nwb', 'bruker', 'movie', 'dcimg'].
- `keep_movie_raw`: Whether to keep binary file of non-registered frames.
- `nplanes`: Each tiff / file has this many planes in sequence.
- `nrois`: Each tiff / file has this many different ROIs.
- `nchannels`: Specify one- or two- channel recording.
- `functional_chan`: The channel to use to extract functional ROIs (1-based).
- `lines`, `dy`, `dx`: Line numbers for each ScanImage ROI within the tiffs, Y position for each ROI, and X position for each ROI.
- `save_path0`: Directory to store results, defaults to `data_path[0]`.
- `fast_disk`: Directory to store temporary binary file (recommended to be SSD), defaults to `save_path0`.

After conversion to binary files, a plane-specific 'db.npy' file is added to each plane folder with plane-specific keys, including:

- `Ly`, `Lx`: Size of the field of view in pixels in Y and X.
- `reg_file`: Location of the binary file, in which registered frames will be saved (before registration, this contains the raw frames).
- `raw_file` (optional): If `keep_movie_raw`, the location of the raw binary file to be registered and saved as `reg_file`.

Motion correction

The settings for motion correction are in the dictionary called 'registration'. The `batch_size` parameter sets the number of frames in each chunk processed together - this may need to be reduced on GPUs with less memory. There is the option to use the anatomical channel for registration, by setting `align_by_chan2` to True. During motion correction, all registered frames are cumulatively averaged, producing a registered mean image, saved in the registration outputs as `meanImg`, and `meanImg_chan2` if there is a second anatomical channel.

Bidirectional phase offset (optional)

If `do_bidiphase` is True, then the bidirectional phase offset is estimated from a subset of frames, with the size of the subset defined by the parameter `nimg_init`. The bidirectional phase offset is defined as the optimal shift between subsequent lines of the raster-scanning, computed via phase-correlation. Alternatively, the user can specify the shift with the setting `bidiphase`. The computed or user-defined shift is then applied before any motion estimates from frames, and when the motion correction shifts are applied to each frame. The shift used is saved in the registration outputs as key `bidiphase`.

Reference image computation

A subset of frames are read in from the entire movie at equal spacing, with the size of the subset defined by the parameter `nimg_init`. The pairwise correlations between these frames is computed (Figure S3b). We choose the frame which has the largest correlation with its 20 most-correlated frames, and average these 20 frames. We use this average as an initial reference image, and compute the phase-correlation (rigid) of all the frames in the subset to this reference image iteratively, across 8 iterations. On each iteration, we average the frames most correlated to the current reference, after applying the estimated shifts from the phase-correlation, to create a new reference. The number of top correlated frames to be averaged is a function of the iteration i defined as $nimg_init * (1 + i) / 16$. This reference image is saved in the registration outputs as key `refImg`.

Rigid registration

The rigid registration step computes the offset in X and Y between each frame and the reference image, and applies these shifts to each frame. To compute the offset, the reference image is first clipped, whitened and smoothed. The clipping, done if `norm_frames` is True (default is True), computes the first and 99th percentile of the pixel intensities in the reference image, and clips the reference image at these values, saved as keys `rmin` and `rmax` in the registration outputs. This clipping is also applied to all frames in the movie before computing the motion shifts. The reference image is next transferred to the Fourier domain and whitened. Filtering by a Gaussian of standard deviation `smooth_sigma` is done directly in the Fourier domain by elementwise multiplication of the Fourier transform of this Gaussian.

Next we apply a spatial tapering on each frame of the movie, so that the phase-correlation is focused on the central part of the field of view rather than the edges. We define the spatial taper as the outer product of two

sigmoids $\mathbf{m}_y$ and $\mathbf{m}_x$ across y and x pixels. The slope of the sigmoid is controlled by setting `spatial_taper` t :

$$m_y(y) = \frac{1}{1 + e^{-(\mu_y + 2t - |y - \mu_y|)/t}}$$

where pixel center μ_y defined as $(L_y - 1)/2$.

Then we whiten each frame and compute the cross-correlation of the reference with each whitened frame; cross-correlation using whitened images is called “phase-correlation” (Figure 2b). The fast FFT operation is greatly accelerated using the GPU.

The peak of the phase-correlation is determined in a range up to a fraction of the frame size, defined by `maxregshift`, to avoid outliers. The peak of the phase-correlation defines the offset in X and Y of the frame with the reference. We shift each frame according to the offsets using the ‘roll’ operation for efficiency - this fills the edges of the frame with the opposing edges during shifts, but we later ignore these regions during cell detection anyway.

Non-rigid registration

After each batch of frames is rigid registered, we optionally compute and apply non-rigid shifts to each frame, if `nonrigid` is True. We recommend applying this step on most recordings, especially if they have a sufficient SNR. One can tell if a recording has sufficient SNR by the outliers of the registration process: if the registration offset traces are jagged and jump to extreme values and back, this is due to low SNR (or possibly due to extreme instability in the recording setup). The non-rigid shifts are computed by performing phase-correlation on square sub-blocks of the frame defined to be overlapping by at least 50% in each dimension, with the size defined by the setting `block_size` (Figure 2d). Each block is spatially tapered with a sigmoidal function with slope parameter $2 \times \text{smooth_sigma}$ and whitened, and the phase-correlation is computed with the reference image in each block (filtered and whitened as in rigid registration). The maximal allowed shift of each block is defined in pixels by the setting `maxregshiftNR`.

Some parts of the image on some frames may have lower SNR - therefore we apply adaptive smoothing of the phase-correlation maps across blocks as a function of SNR. The smoothing filter is a 2D Gaussian with standard deviation equal to one block, that is applied up to two times. The SNR is estimated as the ratio between the peak value of the phase-correlation and the maximum phase-correlation outside the peak region (3x3 pixels). If the SNR is less than the `SNR_thresh` setting, then the smoothed phase-correlation is used. If the once-smoothed phase-correlation still does not have SNR higher than

the `SNR_thresh`, then the twice-smoothed phase-correlation is used.

To estimate the XY shift of each block, we upsample the phase-correlation map by a factor of 10 around the peak (3x3 pixels) using Kriging interpolation with a Gaussian kernel of standard deviation 0.85 pixels. The peak of the upsampled correlation map provides a subpixel estimate of the XY shift for each block. To convert from block-wise shifts to a shift for each pixel in the image, we use bilinear interpolation, assuming the block shifts correspond to the shifts of their center pixels. We then shift the image by the pixel-wise shift map using bilinear interpolation. These interpolation operations are fast to perform on the GPU.

Valid region estimation

Because the frames have moved in X and Y, there are regions on the edges of the frames that are missing information as the putative ROIs have moved outside the field of view. We thus ignore these edge regions for ROI detection. We compute the edge regions using the offsets from the rigid registration, excluding frames with shifts which may be outliers (called `badframes` in the output of the registration, these frames are also excluded from ROI detection and can be user-defined). For this we median filter the XY-offsets and the peak phase-correlation in time with a window of 101 frames. We define large deviations using the Euclidean distance between the XY-offsets and the median-filtered XY-offsets, normalized by the mean distance. We define low correlation frames by the ratio between the peak phase-correlation and the median-filtered peak phase-correlation. If the ratio of these two values is larger than `100*th_badframes`, then the frame is defined as a ‘bad’ frame and excluded from the valid region computation. We also define bad frames to include frames in which the shift was larger than 95% of the maximal allowed rigid shift, defined by `maxregshift`. Excluding the bad frames, the maximum absolute shifts in both X and Y are computed across frames. These pixel shifts define the sizes of the excluded edges on each side of the frame.

Registration metrics

We subsample 2,000-5,000 frames from the registered recording to estimate the accuracy of the motion correction (Figure 2e). We only sample 2,000 frames from the recording if the frame size is larger than 700 (to avoid memory issues) or if there are fewer than 5,000 frames. We crop the frames to the valid frame region and then compute the top 30 principal components of the frames. We average the frames with the 300 smallest weights and the 300 largest weights for each PC (Figure 2f). We register these top

and bottom mean frames to each other to determine the motion shifts within the PC. We return the rigid shift distance, the non-rigid shift distance averaged across blocks, the maximum non-rigid shift distance across blocks, and the rigid and non-rigid shift distances added together and averaged across blocks (shown in Figure 2g, as the ‘mean registration offset’).

ROI detection

The settings for detection are contained in the dictionary ‘detection’. First, using the valid region and excluding the bad frames, the frames of the recording are binned. The bin size is defined as the rounded timescale of the indicator decay, in frames ($fs * \tau$), since the frames within this bin size are likely to be correlated and contain redundant information. We create a binned movie of size up to `nbins`. If the movie is longer than `nbins * fs * tau`, then the bin size is defined as the number of frames divided by `nbins`.

Optionally the binned movie can be PCA-denoised, if `denoise` is True. The PCA denoising is performed in blocks of size `block_size` (registration setting). The denoising is the reconstruction of each block using the top half of the principal components computed per block.

Next the movie is high-pass filtered in time, to reduce slow temporal components such as drift or neuropil, by subtracting the Gaussian filtered version of the movie from itself, with a Gaussian of standard deviation `highpass_time`. After high-pass filtering, the maximum of the movie across time is computed as the maximum projection image, called `max_proj` in the detection outputs.

We provide three algorithms, which can be specified by the `algorithm` setting, as ‘sparsery’, ‘sourcery’, or ‘cellpose’. For Sparsery and Sourcery, we compute the per-pixel standard deviation of the difference between consecutive binned frames in time, which is an estimate of the shot noise per pixel.

After ROI detection, there are optional steps that can remove ROIs before signal extraction. By default, we remove ROIs with more than 75% of their pixels overlapping with other ROIs (specified as `max_overlap = 0.75`). Optionally, the user can specify minimum and/or maximum sizes for removing ROIs, defined by `npix_norm_min` and `npix_norm_max` (see more details about ‘npix_norm’ below). Also, a shape- and size-based classifier can be applied to the ROIs before extraction, with the `preclassify` setting specifying the threshold for ROI inclusion.

Sparsery algorithm

The main detection algorithm, Sparsery, in Suite2p uses functional information to detect ROIs. The

settings for Sparsery are in the ‘sparsery_settings’ dictionary. We first remove the low spatial frequency components from the movie, which are the background neuropil signals (Figure 4a). We divide each binned frame by the per-pixel shot noise estimate and then high-pass filter each binned frame by subtracting the frame uniform filtered with a filter size in pixels of `highpass_neuropil`. The Sparsery algorithm performs a matrix decomposition on this movie, assuming L0-sparse sources in space and L0-sparse traces in time. The cost function is:

$$\text{Cost} = \left(F(x, y, t) - \sum_k M_k(x, y) f_k(t) \right)^2 + \lambda_1 \|M_k\|_0 + \lambda_2 \|f_k\|_0$$

$$\|x\|_0 = \sum_i \delta(x_i)$$

$$\delta(x) = 1, \text{ if } |x| > 0, \text{ and } 0 \text{ otherwise}$$

The effect of this cost function is to introduce a fixed penalty λ_1 when activating a pixel in any ROI mask M_k and another fixed penalty λ_2 when activating a temporal value in any ROI trace f_k . Rather than directly specifying λ_1 and λ_2 , we set them implicitly and adaptively in terms of the thresholds they induce on the activation of non-zero coefficients in the cost function. We optimize this function greedily, one ROI at a time. To initialize ROIs, we consider all possible uniform square templates of sizes 3x3, 6x6, 12x12, 24x24, 48x48. We found these values to cover most if not all sizes of ROIs in typical recordings. These serve as initial guesses for M_k , and they explain a sufficient amount of variance to significantly reduce the cost function and initialize the optimization for M_k . They also have the property that we can efficiently compute the variance explained of each square template via convolution with each frame, followed by thresholding according to λ_2 . For the larger templates, performing this convolution densely would incur large computational penalties. However, the variance explained of larger square templates is highly correlated across space: shifting a square template over by a few pixels from its optimal placement still explains a large amount of variance. Thus, we apply the square templates only at a subset of positions. We devised an efficient algorithm to do so via successive downsampling of the frames by additional factors of 2, followed by convolution with 3x3 uniform filters to exactly calculate the variance explained of templates spaced 2^n pixels apart where n is the downsampling iteration accounting for the square filter sizes $3 \cdot 2^n$.

After constructing these variance explained maps at multiple resolutions, we estimate the scale on which most templates exist. The maximum of each template map across time is interpolated to the original movie size in pixels using bivariate spline approximation of degree 3 with the `RectBivariateSpline` function in `scipy`. Then the maximum across these interpolated maps is computed - is saved as `Vcorr` in the detection outputs and shown in the GUI as the 'correlation map', which can be used to determine if the algorithm may have missed correlated pixel groups. The peaks of this correlation map are found by subtracting off the maximum filtering of the correlation map with a filter size of 11x11 pixels and finding the pixels within 1e-4 of the maximum-filtering. The top 50 peaks, according to their value in the correlation map, are used to determine the best spatial scale, defined as the template map across scales on which the most of these 50 peaks have their maximum. Alternatively, the user can specify the estimated spatial scale (`spatial_scale`) with an integer from 1 to 4 corresponding to templates of size 6x6 to 48x48.

To compute the temporal threshold 'Th2' above which temporal coefficients are activated (corresponding to the λ_2 penalty above), we multiply the inferred spatial scale by a constant: $\text{Th2} = 5 * \text{spatial_scale} * \text{threshold_scaling}$, where the `threshold_scaling` can varied to find more or fewer ROIs. Intuitively, if the same cell is recorded at a higher pixel resolution, there will be more pixels per cells corresponding to a larger overall variance explainable for the same cell, hence the dependence on `threshold_scaling`. Only deviations above the Th2 threshold are used to compute the variance explained of the template (example trace in Figure 4d; variance explained maps in Figure 4b). To extract another cell, we require the square template to cross threshold at least a few times during the recording. This is enforced by setting a minimum variance explained requirement of $\text{Th2} \cdot \max(1, \text{nframes}/1200)$, where `nframes` is the number of binned frames. Thus, if the number of binned frames is relatively small, a single threshold crossing is necessary to initialize a template.

The algorithm iteratively detects ROIs to minimize the cost function. On each iteration, the location of highest variance explained across all template maps and positions is found and used as the initial template (Figure 2c). The active frames for the template are defined as the frames in the movie in which the template surpassed the threshold 'Th2' (Figure 2d). Alternatively, the user can specify a percentile to use instead of a threshold with parameter `active_percentile`, which can produce less sparse

traces. The ROI is extended iteratively by one pixel on each edge, until there are no pixels that can be added (Figure 4e). Pixels are added if their mean intensities across active frames (called 'lam') are larger than the one-fifth of the maximum mean intensity across current ROI pixels; pixels are removed if they are less than one-fifth of the maximum. This procedure implicitly determines λ_1 , though in an adaptive manner for each ROI. The active frames are then recomputed using the newly refined ROI, and this process of iterative refinement is repeated twice more, or discontinued if there are no active frames after refinement. The activity of the ROI on its active frames are subtracted from the movie, and the variance explained of the templates are updated accordingly. The algorithm continues until the highest variance explained is below the threshold described in the previous paragraph, or when the maximum number of ROIs is reached, defined by parameter `max_ROIs`.

On each ROI iteration, there is a check to see if splitting the ROI would improve the variance explained. This check is performed by running iterative k-means, initialized with a split of the ROI into pixels above or below the mean pixel values at the time frame with the largest activation. Each ROI is stored as a dictionary, with the pixels of the ROI saved as 'ypix' and 'xpix' keys. The pixel weights 'lam' are the mean intensity of each pixel during the active frames, multiplied by the previously divided shot noise estimation. The initial template center location is saved in the 'med' key. The initial template map ID is saved in the 'footprint' key.

Note that this procedure is a very greedy, but very efficient optimization of the initial cost function described above. However, the algorithm maps onto intuitive steps that a human might take to find ROIs in such a movie: use a variance explained map to find regions of high unexplained variance, jump to frames where those regions are activated substantially, inspect whether the activated pixels at those frames look like cells, average the profile across frames, then account for the times when this cell is active and look for other masks that may be highly active at different times. Critical to these steps is the L0 interpretation: sources are only interpreted as cells or "good ROIs" if they are strongly active on at least a few frames, and highly spatially localized. Unfortunately, L0 optimization is very difficult, very slow, and always approximate, hence the need to make the compromises we described above.

Sourcery algorithm

Another detection algorithm, Sourcery, also uses functional information to detect ROIs. We describe this algorithm here for completeness, even though it is not

used for the main analyses of the paper. This algorithm can be useful for low SNR data with compact ROIs, as it relies less on sparse, transient deviations from a baseline and more on overall correlations between pixels. The cost function for Sourcery is similar to that above for Sparsery, excluding the temporal sparsity penalty:

$$\text{Cost} = \left(F(x, y, t) - \sum_k M_k(x, y) f_k(t) \right)^2 + \lambda_1 \|M_k\|_0$$

The optimization proceeds in a similar greedy fashion, however without consider temporal sparsity, and replacing some of the updating steps in Sparsery such as the splitting condition and the cell-by-cell update of the cost function. The settings for Sourcery are in the 'sourcery_settings' dictionary. The first step in Sourcery is to orthogonalize the binned movie via PCA (this movie has also been high-pass filtered in time as described above). We also divide each binned frame by the per-pixel shot noise estimate. Then the movie is smoothed spatially by a 2D Gaussian of standard deviation `diameter / 10`. The temporal PCs are computed by applying PCA to the covariance across timepoints, and projected onto the movie to obtain the spatial pixel-wise PCs (called U). The number of top PCs used in the subsequent steps is the minimum between the number of time bins in the binned movie and one-half of the number of total pixels ($X*Y$) (usually the number of bins is smaller).

We model the neuropil as an orthonormal spatial basis functions (S) with low spatial frequency using a sine and cosine basis, and project out this basis from the spatial components of the data U . This is to improve cell detection and to help avoid extracting neuropil components with large spatial extents. The neuropil basis functions are defined as the outer products of sines and cosines, with the frequencies ranging from one to $\lfloor n_y/2 \rfloor$ and $\lfloor n_x/2 \rfloor$ cycles across the field of view, where n_y and n_x are the number of sines and cosines in Y and X respectively. n_y is set as the number of pixels in Y of the FOV divided by the `ratio_neuropil` setting and divided by the `diameter`; n_x is computed the same way using the number of pixels in X. The `ratio_neuropil` setting is approximately the spatial scale of the neuropil relative to the size of the ROIs - in many two-photon recordings this may be around 6. The basis functions are normalized to be unit norm. The neuropil contribution to the PCs (N) is estimated by linear regression, and subtracted from the PCs: $U_{\text{cell}} = U - S(S^T S)^{-1}(S^T U)$.

From this remaining U_{cell} , the cell ROIs are found via an iterative algorithm. U_{cell} is first smoothed with a Gaussian filter with standard deviation of `diameter / 4`, producing U_{smooth} . The sum of squared PC values for each pixel of the smoothed PCs is normalized by the sum of squared PC values from the non-smoothed PCs smoothed by the same Gaussian filter. A morphological opening is performed on this spatial map of radius `diameter / 4` and subtracted off, in order to make the map more uniform across the field of view. This map is like a correlation map, high in areas in which pixels are correlated to each other, and is saved in the detection output as 'Vcorr'. To estimate a threshold on the map values for ROI seeds, we find the local peaks in this map by maximum filtering the map with a 3x3 filter size, and finding all pixels with original values greater than their maximum filtered values. The threshold for the ROI seeds is set as `threshold_scaling` times the median of the peak values.

We next iterate over the maximum local values of the map in groups of 200 ROIs, before recomputing the map. We take the largest value in the map as a seed for the ROI and define its initial PC values ($\mathbf{v}$) as U_{smooth} at the seed. The pixel weights ' λ ' (λ) are defined as the dot product between U_{cell} and $\mathbf{v}$ for each pixel. We iteratively extend the ROI by one pixel on each edge; pixels are removed if they are less than one-fifth of the maximum. The pixel weights are normalized to be unit norm. After each extension, in this first phase, we recompute $\mathbf{v}$ as $U_{\text{cell}} \lambda$. This extension procedure is repeated until no more pixels are added to the ROI. We subtract the predicted ROI values for each pixel $\lambda \mathbf{v}^T$ from U_{cell} , and we set the ROI pixel values (with one surrounding pixel added) in the map to zero. We perform this procedure up to 200 times, finding the next largest value in the map, as long as this value is greater than the threshold.

After each group of ROI detections, we recompute the neuropil PC values N and ROI PC values V . We solve $U = \Lambda V + SN$ via linear regression with an L2 regularizer of size 1e-3. We then update U_{cell} :

$$U_{\text{cell}} = U - SN - \Lambda V.$$

With this updated U_{cell} and V , for each ROI we refine its pixel weights and extend the pixels with the same procedure as above.

ROI detection continues in groups of 200 until the maximum number of iterations `max_iterations` are reached, or there are no more ROIs found. Two more iterations of optimization are added at the end, where the temporal components of the cells are kept fixed, while the spatial components are updated using the un-smoothed PCs of the movie (if parameter

`smooth_masks` is False). This step ensures that the ROI masks do not get over-smoothed. Before these two iterations, the ROIs are reduced to their connected regions if the parameter `connected` is True. The neuropil timecourses are still updated during this step, since the neuropil masks are kept fixed.

Cellpose algorithm

The user can alternatively run an anatomical algorithm to find ROIs. For this we provide Cellpose, a generalist cellular segmentation algorithm [42]. The image used for segmentation is specified by setting `img`, and can be the ratio of the maximum projection image and the mean image (`'max_proj / meanImg'`), the mean image (`'meanImg'`), or the maximum projection image (`'max_proj'`). This image can be high-pass filtered before running Cellpose, with setting `highpass_spatial`. The user can specify the Cellpose model with setting `cellpose_model`.

The pixel weights in each ROI `'lam'` are the pixel values from the maximum projection image if `'max_proj / meanImg'` or `'max_proj'` are used as the segmentation. If the mean image is used, then the weights are the mean image clipped at the first and 99th percentile and scaled between 0.1 and 1.1. The `'med'` center of the ROI is the median of the pixels in the ROI, or the pixel within the ROI closest to the median if the median is outside the ROI.

Signal extraction

Next we extract the time course for each ROI and the corresponding neuropil signal for each ROI. The settings for signal extraction are contained in the dictionary `'extraction'`. ROIs can be overlapping (see example in Figure 4f). By default we do not use pixels that are in multiple ROIs for signal extraction, but this can be turned off by setting `allow_overlap` to False. The ROI mask values are defined using the `'lam'` weights for each pixel, normalized to sum to 1 for each ROI.

The neuropil is the signal from the dendrites and axons surrounding the ROI that are captured within its point spread function, and it can be estimated by using the activity surrounding the ROI [59]. We define the neuropil masks as squares around each ROI, with a minimum size `min_neuropil_pixels` and with a padding around the ROI of `inner_neuropil_radius` in pixels. Other ROI pixels are excluded from the neuropil masks. In fields of view with many ROIs, this may exclude many pixels and reduce the accuracy of the neuropil estimate. Thus we do not exclude all ROI pixels and only exclude pixels with sufficiently high `'lam'` weights, determined by an adaptive percentile threshold. The percentile filter is applied in a window of

five times the median ROI radius with a percentile set by `lam_percentile`, and the pixels less than the filter output are not excluded from the neuropil estimation. The neuropil mask weights are defined to be equal in size, as one over the number of pixels in the mask.

To extract the ROI and neuropil traces, a matrix multiplication is performed between the ROI and neuropil masks and the registered frames, in batches defined by the setting `batch_size`. This matrix multiplication is a sparse matrix multiplication that is accelerated on the GPU. The outputs are defined as `'F'` and `'Fneu'` for the ROI and neuropil traces respectively.

The default neuropil coefficient is 0.7, and is specified by the setting `neuropil_coefficient`. Before spike deconvolution, the neuropil trace is subtracted from the ROI trace with the neuropil coefficient. Then an additive baseline is estimated for each trace. The settings for the baselining are in the dictionary `'dcnv_preprocess'`. First the trace is smoothed by a Gaussian filter of standard deviation `sig_baseline` frames. In the default baseline computation, `'maximin'`, the rolling maximum of the rolling minimum is computed in a window of `win_baseline` seconds. This baseline is then subtracted from the trace, and then the trace is deconvolved using OASIS with no sparsity constraints [13]. OASIS requires the input of the timescale of the indicator decay, which is defined in the pipeline as `tau * fs`. We accelerated the OASIS deconvolution using `numba`, in batches of frames of size `batch_size` [65].

ROI statistics and classification

We compute several statistics of the ROI shape and the neuropil-corrected fluorescence trace, which are saved in the dictionary for each ROI, which contains `'ypix'`, `'xpix'`, `'lam'` and `'npix'` as defined during ROI extraction:

- `'soma_crop'`: We estimate the pixels in the somatic compartment of the ROI by computing the sum of pixel weights (`'lam'`) within a radius of the center `'med'` of the ROI. We compute the difference of this sum as a function of the radius. The radius at which this difference is less than one-third of its maximum value is defined as the radius of the somatic compartment of the ROI, if this radius exists - otherwise all pixels are considered to be somatic. The `'soma_crop'` key is a boolean list with True values for the pixels in the somatic compartment. The total number of soma pixels is saved in `'npix_soma'`. Note: for all the following shape estimates, only the somatic pixels are used - the user can turn off the somatic cropping by setting `soma_crop` to False in the `'detection'` settings.
- `'compact'`: We compute how compact an ROI is,

where a compactness of 1.0 is equivalent to a disk, and larger values mean the ROI is less compact than a disk. We compute the median pixel of 'ypix' and 'xpix', and compute the mean Euclidean distance of each pixel from this median (saved as 'mrs'). This disk normalized by the mean Euclidean distance for the pixels in a disk ROI with the same number of pixels (saved as 'mrs0').

- 'radius': The radius of the ROI, estimated as two times the major axis radius of a multivariate Gaussian fit to the pixel weights.
- 'aspect_ratio': The ratio between two times the major axis radius and the sum of the major and minor axis radii from the multivariate Gaussian fit. This produces an aspect ratio between 1.0 and 2.0 where larger values indicate more elongated ROIs.
- 'npix_norm': The number of pixels 'npix_soma' are normalized by the the median size of the first 100 ROIs extracted (largest variance explained), or by the median size of all ROIs if Cellpose is used for extraction. 'npix_norm_no_crop' uses 'npix' for this computation instead.
- 'overlap': boolean array specifying which pixels in the ROI are overlapping with other ROIs.
- 'snr': An estimate of the SNR of the activity trace x , defined as

$$\text{SNR} = 1 - \frac{\text{var}_t[x_t - x_{t-1}]}{2 \text{var}_t[x_t]}$$

(similar to [55]).

- 'skew': The skewness of the activity trace.
- 'std': The standard deviation of the activity trace.

The ROI classifier in Suite2p by default uses the 'skew', 'npix_norm', and 'compact' features. If applied before signal extraction (with `preclassify`), only 'npix_norm' and 'compact' feature are used. The classifier is trained on ground-truth labels of what is a cell and not a cell (saved in 'iscell'), which the user can label in the GUI. The classifier is a type of weighted, non-parametric, naive Bayes classifier [78]. Each feature is non-parametrically binned into 100 bins separately for the cell and not-cell classes. The probabilities in each bin are smoothed across bins by a Gaussian with standard deviation of 2 bins. The log posterior likelihood of each feature is then used as input to a logistic regression classifier that is optimized with the 'liblinear' solver, using an L2 regularization of

size 100 [66]. Effectively, this secondary classification step weighs the importance of each feature relative to the others. A simple naive Bayes classifier would simply use weights of 1 for each feature axis.

Benchmarking motion correction

To benchmark motion correction, we used a recording in which neurons expressed both a functional and an anatomical marker (GCaMP6s and td-Tomato), recorded in separate imaging channels (Figure 3a). This recording had a resolution of 1 pixel per μm , fields of view of size 1112 x 650 pixels, a recording rate of 6.76 Hz, and a recording length of 80 minutes. We applied the motion correction algorithms to the recording, using the functional channel to compute the motion shifts and applying these shifts to the anatomical channel. The Suite2p motion correction pipeline was run with the defaults. The Caiman motion correction was run with the defaults, other than the maximum rigid shifts, `max_shifts`, and the maximum non-rigid shifts, `max_deviation_rigid`, which were set to (65,65) and 10 respectively, which were an upper bound on the deviations in this dataset, and which matched the defaults in Suite2p. We also updated the Fiola parameter for maximum shifts `ms` to [65, 65], but otherwise kept the parameters as the defaults. We computed the registration metrics for the functional and anatomical channels after registration with each algorithm (Figure 3b-f).

For quantifying timing performance, we ran each algorithm on a subset of the tiffs, from 500 to 32,000 frames (Figure 3g). The Caiman algorithm was run on 16 cores on an Emerald Rapids compute node. The Suite2p and Fiola algorithms were run on 12 cores on a Cascade Lake compute node with an A100 GPU. The frame processing rate in Hz was estimated as the slope of the number of frames as a function of the runtime.

Benchmarking detection

Hybrid ground-truth generation

To benchmark detection, we generated hybrid ground-truth recordings (Figure 5a). For this, we recorded three mice expressing RiboL1-jGCaMP8s so that there were no background neuropil signals [40]. The recordings were at a pixel resolution of 1 pixel per μm , with four planes spaced 30 μm apart, a frame rate of 15-15.5Hz, a frame size that varied slightly across the three mice due to manual selection of the recording field of view (744x654, 726x654, and 748x652), and a recording length of 60-66 minutes. We computed the maximum projection image (maximum across frames in time) for each recorded plane, and extracted the "ground-truth" ROIs from this image using the Cellpose SAM model [79]. The Cellpose

SAM model was run with settings `diameter=16`, `cellprob_threshold=-3`, and `flow_threshold=0`. From each detected ROI, the fluorescence trace for the ROI and the neuropil signal were extracted using default Suite2p settings. The neuropil-corrected traces were then used as the ground-truth traces (the neuropil correction is performed to remove frame-wide changes in brightness).

For the hybrid ground-truth recording, we downsampled the recording by a factor of two spatially, so that the pixel per μm was now 2, more similar to many imaging setups. This downsampled recording is the base onto which we added simulated axons/dendrites, simulated neuropil, and shot noise. Accordingly, the ground-truth ROIs were also downsampled spatially by a factor of two for comparison with the detected ROIs.

For the simulated axons/dendrites and simulated neuropil, we removed slow fluctuations in time from the recording. This was performed by estimating a per pixel baseline by maximum filtering of a minimum filter of the recording, with a filter width of 1 second, and then subtracting this baseline from each pixel. After this filtering, we extracted fluorescence traces for each Cellpose ROI, which were used for the axons/dendrites. For the simulated neuropil, we downsampled the filtered recordings by a factor of two spatially, and also subtracted the per pixel mean from the recording, using the mean pixel intensities from the first 2,000 frames.

The simulated axon/dendrite ROIs were ellipses with centers drawn from a uniform random distribution across the field of view (x_0, y_0) - the default number of ellipses was set to 2,000. The ellipse radii, for one axis of the ellipse, were drawn from a uniform distribution between 0.75 and 2. For the radii of the other axis, two-thirds of the ellipses had radii drawn from a uniform distribution between 0.75 and 2, and one-third had radii drawn from a uniform distribution between 1 and 6. For the pixel weights 'lam' of each ellipse ROI, we added Gaussian random noise to the output of the ellipse equation for each pixel, $l_e = \frac{(y-y_0)^2}{a^2} + \frac{(x-x_0)^2}{b^2}$, and rectified the values (set the minimum to 0). The 'lam' pixel weights were then set to $(1 - l_e) / (1 - \bar{l}_e)$, so that the largest weights were at the center of the ROI, and that the weights summed to the number of pixels in the ROI. We varied the number of such ROIs (Figure 5d left), keeping the other parameters at their default values. For the fluorescence traces, we sampled activity traces with an SNR > 0.3 which were nearby spatially on the other three planes: we computed the Euclidean distance between the ellipse ROI center and the ROIs on the other planes, and used the activity trace from the n^{th} closest ROI, where

n was drawn from a random exponential of mean five and rounded to an integer. These activity traces were added to the recording, weighted per pixel by the 'lam' weights of the ROI.

The simulated neuropil consisted of 7x7 basis functions in X and Y, defined as the outer product of sines and cosines, with the frequencies from one to three cycles across the field of view. Each basis function was normalized to be unit norm. Then the activity of each basis function was simulated by projecting the basis function onto the filtered and downsampled recordings from the other three planes, summed across the planes. Then this activity was projected onto the basis functions to create the simulated neuropil. After projection, the per pixel mean from the downsampled recording without filtering was added, and then the neuropil activity was rectified (minimum set to 0). Then the neuropil activity was scaled by the neuropil fraction (default 0.4) and added to the recording, which was scaled by one minus the neuropil fraction. We varied the neuropil fraction (Figure 5d middle), while keeping the other parameters at their default values.

Next we simulated shot noise using a Poisson distribution, with a scaling term, the Poisson noise scale, which was set to 20 by default. The recording was divided by the Poisson noise scale then used as the mean of a Poisson distribution, which we sampled from to generate the intensity of each pixel across all timepoints in the recording. The output was then multiplied by the Poisson noise scale to ensure the mean pixel intensity was similar across scales. We varied the Poisson noise scale (Figure 5d right), while keeping the other parameters at their default values.

Performance metrics

We compared the ground-truth ROIs and the detected ROIs, using their pixel distributions and their activity traces. We excluded ROIs with an SNR less than 0.25, assuming these may be non-active or noisy ROIs detected by Cellpose or the other algorithms. We also applied a size filter on the ground-truth and detected ROIs, excluding ROIs smaller than the fifth percentile or larger than the 95th percentile of ground-truth ROI sizes.

We computed the intersection-over-union (IoU) of the detected ROIs and the ground-truth ROIs, which is the number of overlapping pixels divided by the number of unique pixels across both ROIs. If the IoU value was larger than 0.5, then the ROIs were considered to be matched spatially. For the ROIs that were spatially matched, we computed the correlation of their activity traces, and found the most-correlated ground-truth ROI for each detected ROI. If

this correlation was larger than 0.5, then we considered the ROIs to be matched. We next filtered the matched ROIs so that each ground-truth ROI could only be matched to one detected ROI. The remaining matched ROIs were defined as the “true positives” (TP). The “false positives” (FP) were the detected ROIs without matched ground-truth ROIs, and the “false negatives” (FN) were the ground-truth ROIs without matched detected ROIs. The numbers of TP, FP, and FN ROIs were computed for each plane in each recording, and summarized by the F1 score: $\frac{TP}{TP+0.5(FP+FN)}$.

Detection algorithms

Suite2p was run with all default settings, with `tau` set to 0.25 because the indicator was jRCaMP8s. We performed a sweep across the `threshold_scaling` parameter, from 0.6 to 1.4, to determine the optimal threshold for ROI detection, defined by F1 performance averaged across all recordings (Figure S5a). The optimal threshold was determined to be 0.7, and this was used for all benchmarking in Figure 5.

Caiman was run with the settings used for a V1 recording with a 2 μm /pixel spacing from their paper (called ‘yuste_params’). We performed a sweep across the `K` parameter, which specifies the number of components per patch, from 7 to 13 (Figure S5b). The optimal `K` was determined to be 9, and this was used for all benchmarking in Figure 5.

We also performed a grid search over several of the parameters in Caiman, centered at the parameter values used in Figure 5: autoregressive order (`p`), number of background components (`gnb`), number of components (`K`), Gaussian smoothing (`gSig`), and patch half-size (`rf`). We did not observe substantial improvement in the performance of Caiman when varying these parameters (average F1 score of 0.44 versus 0.49 for best parameter combination). In Figure S6, we plotted the parameter combination that produced the highest F1 score for each data point.

The Caiman algorithm was run on 16 cores on a compute node, and the Suite2p algorithm was run on 12 cores on a compute node with an A100 GPU.

References

- [1] Jason N. D. Kerr and Winfried Denk. Imaging in vivo: Watching the brain in action. *Nature Reviews Neuroscience*, 9(3):195–205, March 2008.
- [2] Christine Grienberger and Arthur Konnerth. Imaging calcium in neurons. *Neuron*, 73(5):862–885, 2012.
- [3] Emre Yaksi and Rainer W. Friedrich. Reconstruction of firing rate changes across neuronal populations by temporally deconvolved Ca²⁺ imaging. *Nature Methods*, 3(5):377–383, May 2006.
- [4] Kazuhiro Sohya, Katsuro Kameyama, Yuchio Yanagawa, Kunihiko Obata, and Tadaharu Tsumoto. GABAergic Neurons Are Less Selective to Stimulus Orientation than Excitatory Neurons in Layer II/III of Visual Cortex, as Revealed by In Vivo Functional Ca²⁺ Imaging in Transgenic Mice. *Journal of Neuroscience*, 27(8):2145–2149, February 2007.
- [5] James H. Marshel, Marina E. Garrett, Ian Nauhaus, and Edward M. Callaway. Functional Specialization of Seven Mouse Visual Cortical Areas. *Neuron*, 72(6):1040–1054, December 2011.
- [6] John B. Issa, Benjamin D. Haeffele, Amit Agarwal, Dwight E. Bergles, Eric D. Young, and David T. Yue. Multiscale Optical Ca²⁺ Imaging of Tonal Organization in Mouse Auditory Cortex. *Neuron*, 83(4):944–959, August 2014.
- [7] Laura N Driscoll, Noah L Pettit, Matthias Minderer, Selmaan N Chettih, and Christopher D Harvey. Dynamic reorganization of neuronal activity patterns in parietal cortex. *Cell*, 170(5):986–999, 2017.
- [8] Liron Sheintuch, Alon Rubin, Noa Brande-Eilat, Nitzan Geva, Noa Sadeh, Or Pinchasof, and Yaniv Ziv. Tracking the Same Neurons across Multiple Days in Ca²⁺ Imaging Data. *Cell Reports*, 21(4):1102–1115, October 2017.
- [9] Misha B Ahrens, Michael B Orger, Drew N Robson, Jennifer M Li, and Philipp J Keller. Whole-brain functional imaging at cellular resolution using light-sheet microscopy. *Nature methods*, 10(5):413, 2013.
- [10] Carsen Stringer, Marius Pachitariu, Nicholas Steinmetz, Charu Bai Reddy, Matteo Carandini, and Kenneth D. Harris. Spontaneous behaviors drive multidimensional, brainwide activity. *Science*, 364(6437):255–255, 2019.
- [11] Daniel Aharoni, Baljit S Khakh, Alcino J Silva, and Peyman Golshani. All the light that we can see: A new era in miniaturized microscopy. *Nature methods*, 16(1):11–13, 2019.
- [12] Weijian Zong, Horst A. Obenaus, Emilie R. Skytøen, Hanna Eneqvist, Nienke L. de Jong, Ruben Vale, Marina R. Jorge, May-Britt Moser, and Edvard I. Moser. Large-scale two-photon calcium imaging in freely moving mice. *Cell*, 185(7):1240–1256.e30, March 2022.
- [13] Johannes Friedrich, Pengcheng Zhou, and Liam Paninski. Fast online deconvolution of calcium

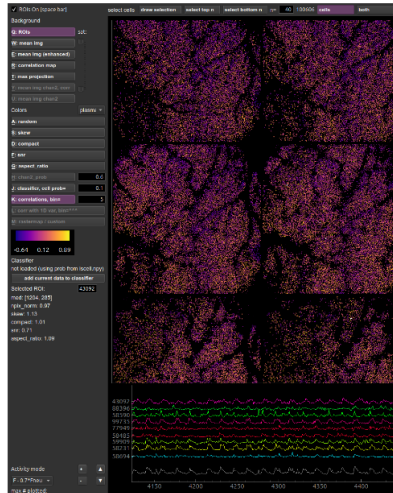
- imaging data. *PLOS Computational Biology*, 13(3):e1005423, 2017.
- [14] E. A. Mukamel, A. Nimmerjahn, and M. J. Schnitzer. Automated Analysis of Cellular Signals from Large-Scale Calcium Imaging Data. *Neuron*, 63:747–760, 2009.
- [15] Ryuichi Maruyama, Kazuma Maeda, Hajime Moroda, Ichiro Kato, Masashi Inoue, Hiroyoshi Miyakawa, and Toru Aonishi. Detecting cells using non-negative matrix factorization on calcium imaging data. *Neural Networks*, 55:11–19, 2014.
- [16] Patrick Kaifosh, Jeffrey D Zaremba, Nathan B Danielson, and Attila Losonczy. Sima: Python software for analysis of dynamic fluorescence imaging data. *Frontiers in neuroinformatics*, 8:80, 2014.
- [17] Ferran Diego Andilla and Fred A Hamprecht. Sparse space-time deconvolution for calcium image analysis. In *Advances in Neural Information Processing Systems*, pages 64–72, 2014.
- [18] Eftychios A Pnevmatikakis, Daniel Soudry, Yuanjun Gao, Timothy A Machado, Josh Merel, David Pfau, Thomas Reardon, Yu Mu, Clay Lacefield, Weijian Yang, et al. Simultaneous denoising, deconvolution, and demixing of calcium imaging data. *Neuron*, 89(2):285–299, 2016.
- [19] Aleksander Klibisz, Derek Rose, Matthew Eicholtz, Jay Blundon, and Stanislav Zakharenko. Fast, simple calcium imaging segmentation with fully convolutional networks. In *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, pages 285–293. Springer, 2017.
- [20] Quico Spaen, Roberto Asín-Achá, Selmaan N Chettih, Matthias Minderer, Christopher Harvey, and Dorit S Hochbaum. Hnccorr: a novel combinatorial approach for cell identification in calcium-imaging movies. *eneuro*, 6(2), 2019.
- [21] Stephanie Reynolds, Therese Abrahamsson, Renaud Schuck, P. Jesper Sjöström, Simon R. Schultz, and Pier Luigi Dragotti. ABLE: An Activity-Based Level Set Segmentation Algorithm for Two-Photon Calcium Imaging Data. *eNeuro*, 4(5), September 2017.
- [22] Ashley Petersen, Noah Simon, and Daniela Witten. SCALPEL: Extracting neurons from calcium imaging data. *The Annals of Applied Statistics*, 12(4):2430–2456, December 2018.
- [23] Pengcheng Zhou, Shanna L Resendez, Jose Rodriguez-Romaguera, Jessica C Jimenez, Shay Q Neufeld, Andrea Giovannucci, Johannes Friedrich, Eftychios A Pnevmatikakis, Garret D Stuber, Rene Hen, Mazen A Kheirbek, Bernardo L Sabatini, Robert E Kass, and Liam Paninski. Efficient and accurate extraction of in vivo calcium signals from microendoscopic video data. *eLife*, 7:e28728, February 2018.
- [24] Somayyeh Soltanian-Zadeh, Kaan Sahingur, Sarah Blau, Yiyang Gong, and Sina Farsiu. Fast and robust active neuron segmentation in two-photon calcium imaging using spatiotemporal deep learning. *Proceedings of the National Academy of Sciences*, 116(17):8554–8563, April 2019.
- [25] Elke Kirschbaum, Alberto Bailoni, and Fred A. Hamprecht. DISCo: Deep Learning, Instance Segmentation, and Correlations for Cell Segmentation in Calcium Imaging. In Anne L. Martel, Purang Abolmaesumi, Danail Stoyanov, Diana Mateus, Maria A. Zuluaga, S. Kevin Zhou, Daniel Racoceanu, and Leo Joskowicz, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020*, pages 151–162, Cham, 2020. Springer International Publishing.
- [26] Adam S. Charles, Nathan Cermak, Rifqi O. Affan, Benjamin B. Scott, Jackie Schiller, and Gal Mishne. GraFT: Graph Filtered Temporal Dictionary Learning for Functional Neural Imaging. *IEEE Transactions on Image Processing*, 31:3509–3524, 2022.
- [27] Fatih Dinç, Hakan Inan, Oscar Hernandez, Claudia Schmuckermair, Omer Hazon, Tugce Tasci, Biafra O. Ahanonu, Yanping Zhang, Jérôme Lecoq, Simon Haziza, Mark J. Wagner, Murat A. Erdogdu, and Mark J. Schnitzer. Fast, scalable, and statistically robust cell extraction from large-scale neural calcium imaging datasets, August 2024.
- [28] Carsen Stringer, Lin Zhong, Atika Syeda, Fengtong Du, Maria Kesa, and Marius Pachitariu. Rastermap: A discovery method for neural population recordings. *Nature Neuroscience*, pages 1–12, 2024.
- [29] Marius Pachitariu, Carsen Stringer, Sylvia Schröder, Mario Dipoppa, L Federico Rossi, Matteo Carandini, and Kenneth D Harris. Suite2p: beyond 10,000 neurons with standard two-photon microscopy. *bioRxiv*, 2016.
- [30] Tim Wang, Marius Pachitariu, Caiying Guo, Albert K Lee, Jae Sung Lee, Yan Zhang, Marton Rozsa, Jeremy Hasseman, Ilya Kolb, Jim D. Cox, Crystal Lopez, Loren Looger, and Karel Svoboda. jGCaMP8 transgenic mice. *figshare*, 11 2023.

- [31] David S Greenberg and Jason ND Kerr. Automated correction of fast motion artifacts for two-photon imaging of awake animals. *Journal of neuroscience methods*, 176(1):1–15, 2009.
- [32] Eftychios A Pnevmatikakis and Andrea Giovannucci. Normcorre: An online algorithm for piecewise rigid motion correction of calcium imaging data. *Journal of neuroscience methods*, 291:83–94, 2017.
- [33] Andrea Giovannucci, Johannes Friedrich, Pat Gunn, Jérémie Kalfon, Brandon L Brown, Sue Ann Koay, Jiannis Taxidis, Farzaneh Najafi, Jeffrey L Gauthier, Pengcheng Zhou, et al. Caiman an open source tool for scalable calcium imaging data analysis. *elife*, 8:e38173, 2019.
- [34] Changjia Cai, Cynthia Dong, Johannes Friedrich, Marton Rozsa, Eftychios A. Pnevmatikakis, and Andrea Giovannucci. FIOLA: An accelerated pipeline for fluorescence imaging online analysis. *Nature Methods*, 20(9):1417–1425, September 2023.
- [35] Eftychios A Pnevmatikakis and Andrea Giovannucci. Normcorre: An online algorithm for piecewise rigid motion correction of calcium imaging data. *bioRxiv*, page 108514, 2017.
- [36] Alfonso Alba, J Flavio Viguera-Gomez, Edgar R Arce-Santana, and Ruth M Aguilar-Ponce. Phase correlation with sub-pixel accuracy: a comparative study in 1d and 2d. *Computer Vision and Image Understanding*, 137:76–87, 2015.
- [37] Hassan Foroosh, Josiane B Zerubia, and Marc Berthod. Extension of phase correlation to subpixel registration. *Image Processing, IEEE Transactions on*, 11(3):188–200, 2002.
- [38] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, November 2005.
- [39] Carsen Stringer and Marius Pachitariu. Computational processing of neural recordings from calcium imaging data. *Current opinion in neurobiology*, 55:22–31, 2019.
- [40] Sverre Grødem, Ingeborg Nymoen, Guro Helén Vatne, Frederik Sebastian Rogge, Valgerður Björnsdóttir, Kristian Kinden Lensjø, and Marianne Fyhn. An updated suite of viral vectors for in vivo calcium imaging using intracerebral and retro-orbital injections in male mice. *Nature Communications*, 14(1):608, 2023.
- [41] SG Mallat and Z Zhang. Matching pursuits with time-frequency dictionaries. *IEEE Transactions on Signal Processing*, 41(12):3397–3415, 1993.
- [42] Carsen Stringer, Tim Wang, Michalis Michaelos, and Marius Pachitariu. Cellpose: a generalist algorithm for cellular segmentation. *Nature methods*, 18(1):100–106, 2021.
- [43] Marius Pachitariu, Carsen Stringer, and Kenneth D. Harris. Robustness of spike deconvolution for neuronal calcium imaging. *Journal of Neuroscience*, 38(37):7976–7985, 2018.
- [44] Thomas A Pologruto, Bernardo L Sabatini, and Karel Svoboda. Scanimage: flexible software for operating laser scanning microscopes. *Biomedical engineering online*, 2(1):13, 2003.
- [45] Joshua T Vogelstein, Adam M Packer, Timothy A Machado, Tanya Sippy, Baktash Babadi, Rafael Yuste, and Liam Paninski. Fast nonnegative deconvolution for spike train inference from population calcium imaging. *Journal of neurophysiology*, 104(6):3691–3704, 2010.
- [46] Kaspar Podgorski and Kurt Haas. Fast non-negative temporal deconvolution for laser scanning microscopy. *Journal of biophotonics*, 6(2):153–162, 2013.
- [47] Jon Onativia, Simon R Schultz, and Pier Luigi Dragotti. A finite rate of innovation algorithm for fast and accurate spike detection from two-photon calcium imaging. *Journal of neural engineering*, 10(4):046017, 2013.
- [48] Stephanie Reynolds, Jon Onativia, Caroline S Copeland, Simon R Schultz, and Pier Luigi Dragotti. Spike detection using FRI methods and protein calcium sensors: performance analysis and comparisons. In *Sampling Theory and Applications (SampTA), 2015 International Conference on*, pages 533–537. IEEE, 2015.
- [49] Thomas Deneux, Attila Kaszas, Gergely Szalay, Gergely Katona, Tamás Lakner, Amiram Grinvald, Balázs Rózsa, and Ivo Vanzetta. Accurate spike estimation from noisy calcium signals for ultrafast three-dimensional imaging of large neuronal populations in vivo. *Nature Communications*, 7, 2016.
- [50] Sean Jewell and Daniela Witten. Exact spike train inference via ℓ_0 optimization. *The annals of applied statistics*, 12(4):2457, 2018.
- [51] Jérôme Tubiana, Sébastien Wolf, and Georges Debregeas. Blind sparse deconvolution for inferring spike trains from fluorescence recordings. *bioRxiv*, page 156364, 2017.
- [52] Abbas Kazempour, Ji Liu, Krystyna Solarana, Daniel A Nagode, Patrick O Kanold, Min Wu, and Behtash Babadi. Fast and stable signal deconvolution via compressible state-space models. *IEEE Transactions on Biomedical Engineering*, 65(1):74–86, 2018.

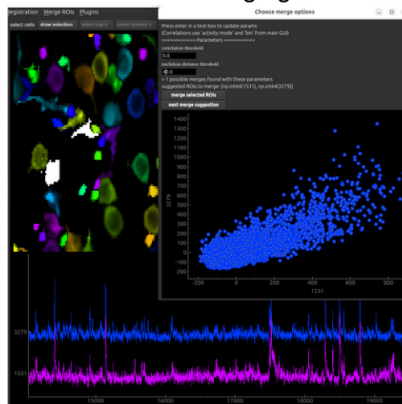
- [53] L. Theis, P. Berens, E. Froudarakis, J. Reimer, M. Roman-Roson, T. Baden, T. Euler, A. S. Tolias, and M. Bethge. Benchmarking spike rate inference in population calcium imaging. *Neuron*, 90(3):471–482, May 2016.
- [54] Artur Speiser, Jinyao Yan, Evan W Archer, Lars Buesing, Srinivas C Turaga, and Jakob H Macke. Fast amortized inference of neural activity from calcium imaging data with variational autoencoders. In *Advances in Neural Information Processing Systems*, pages 4027–4037, 2017.
- [55] Peter Rupperecht, Stefano Carta, Adrian Hoffmann, Mayumi Echizen, Antonin Blot, Alex C. Kwan, Yang Dan, Sonja B. Hofer, Kazuo Kitamura, Fritjof Helmchen, and Rainer W. Friedrich. A database and deep learning toolbox for noise-optimized, generalized spike inference from calcium imaging. *Nature Neuroscience*, 24(9):1324–1337, September 2021.
- [56] Jilt Sebastian, Mriganka Sur, Hema A. Murthy, and Mathew Magimai-Doss. Signal-to-signal neural networks for improved spike estimation from calcium imaging data. *PLOS Computational Biology*, 17(3):e1007921, March 2021.
- [57] Zhanhong Zhou, Hei Matthew Yip, Katya Tsimring, Mriganka Sur, Jacque Pak Kan Ip, and Chung Tin. Effective and efficient neural networks for spike inference from in vivo calcium imaging. *Cell Reports Methods*, 3(5), May 2023.
- [58] Anna S Goncharova, Alf Honigsmann, Florian Jug, and Alexander Krull. Improving blind spot denoising for microscopy. In *European Conference on Computer Vision*, pages 380–393. Springer, 2020.
- [59] Tsai-Wen Chen, Trevor J Wardill, Yi Sun, Stefan R Pulver, Sabine L Renninger, Amy Baohan, Eric R Schreiter, Rex A Kerr, Michael B Orger, Vivek Jayaraman, and others. Ultrasensitive fluorescent proteins for imaging neuronal activity. *Nature*, 499(7458):295–300, 2013.
- [60] Hod Dana, Yi Sun, Boaz Mohar, Brad K. Hulse, Aaron M. Kerlin, Jeremy P. Hasseman, Getahun Tsegaye, Arthur Tsang, Allan Wong, Ronak Patel, John J. Macklin, Yang Chen, Arthur Konnerth, Vivek Jayaraman, Loren L. Looger, Eric R. Schreiter, Karel Svoboda, and Douglas S. Kim. High-performance calcium sensors for imaging activity in neuronal populations and microcompartments. *Nature Methods*, 16(7):649–657, July 2019.
- [61] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.
- [62] Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. The numpy array: a structure for efficient numerical computation. *Computing in Science & Engineering*, 13(2):22, 2011.
- [63] Eric Jones, Travis Oliphant, Pearu Peterson, et al. SciPy: Open source scientific tools for Python, 2001.
- [64] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [65] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, page 7. ACM, 2015.
- [66] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [67] Christoph Gohlke. cgohlke/tifffile: v2022.5.4 (v2022.5.4), 2022.
- [68] Nathan Clack, Lawrence Niu, and Vidrio Technologies. Scanimage tiff reader: Python tiff reader for scanimage. <https://pypi.org/project/scanimage-tiff-reader/>, 2023. Python package version 1.4.1.4.
- [69] Mark Summerfield. *Rapid GUI Programming with Python and Qt: The Definitive Guide to PyQt Programming (paperback)*. Pearson Education, 2007.
- [70] Luke Campagnola. Scientific graphics and gui library for python.
- [71] Talley Lambert. superqt. <https://github.com/pyapp-kit/superqt>, 2023.
- [72] John D Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(3):90, 2007.
- [73] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley,

- Jessica B Hamrick, Jason Grout, Sylvain Corlay, et al. Jupyter notebooks-a publishing format for reproducible computational workflows. In *ELPUB*, pages 87–90, 2016.
- [74] Carsen Stringer, Michalis Michaelos, Dmitri Tsybouski, Sarah E Lindo, and Marius Pachitariu. High-precision coding in visual cortex. *Cell*, 184(10):2767–2778, 2021.
- [75] Qingqing Zhang, Sverre Grødem, Alexa Gracias, Kristian K. Lensjø, Marianne Fyhn, Carsen Stringer, and Marius Pachitariu. Spatial predictive coding in visual cortical neurons, September 2025.
- [76] Nicholas James Sofroniew, Daniel Flickinger, Jonathan King, and Karel Svoboda. A large field of view two-photon mesoscope with subcellular resolution for in vivo imaging. *elife*, 5:e14472, 2016.
- [77] Dmitri Tsybouski, Natalia Orlova, Peter Ledochowitsch, and Peter Saggau. Two-photon frequency division multiplexing for functional in vivo imaging: A feasibility study. *Optics Express*, 27(4):4488–4503, February 2019.
- [78] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [79] Marius Pachitariu, Michael Rariden, and Carsen Stringer. Cellpose-SAM: Superhuman generalization for cellular segmentation. *bioRxiv*, 2025.

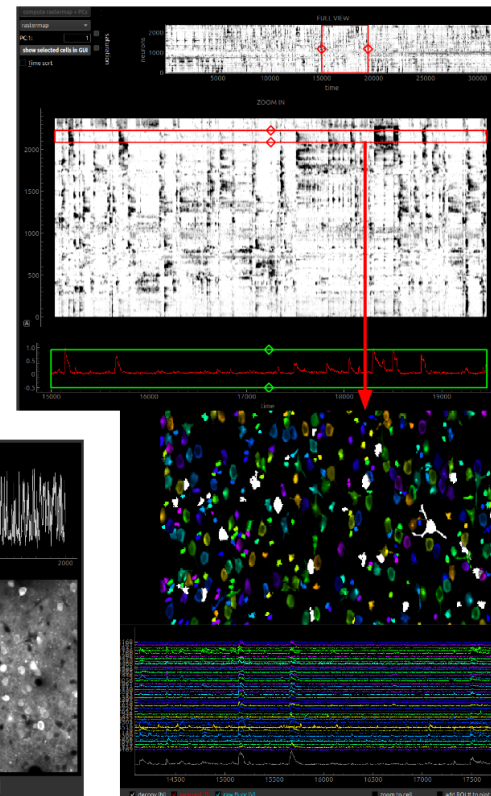
Neural correlation visualization



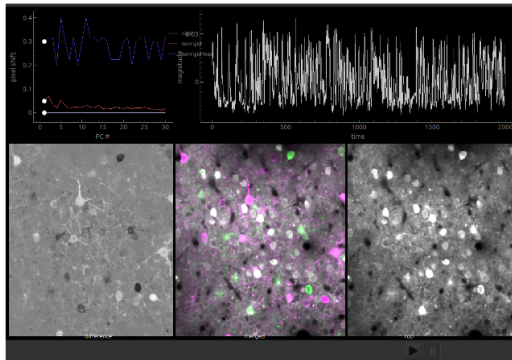
Automated ROI merging



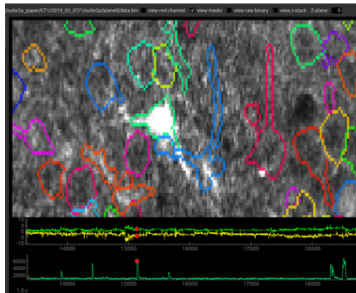
Rastermap visualization and selection of neuron clusters



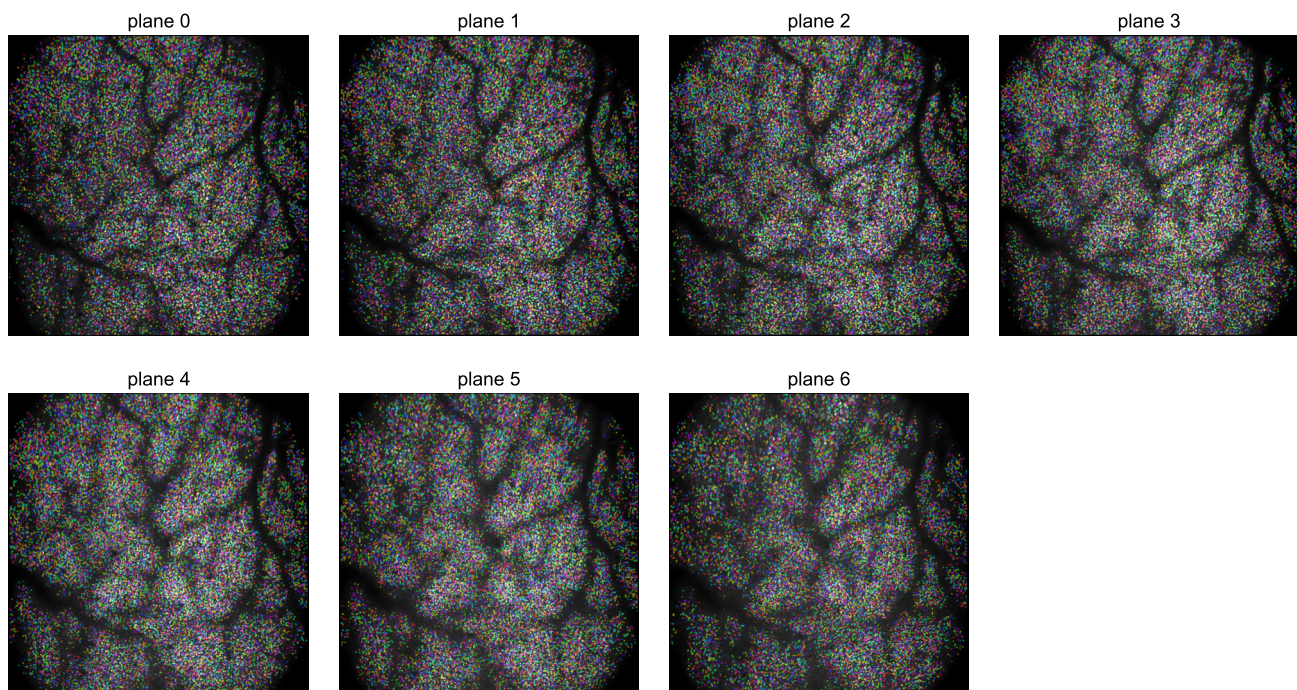
Registration metrics



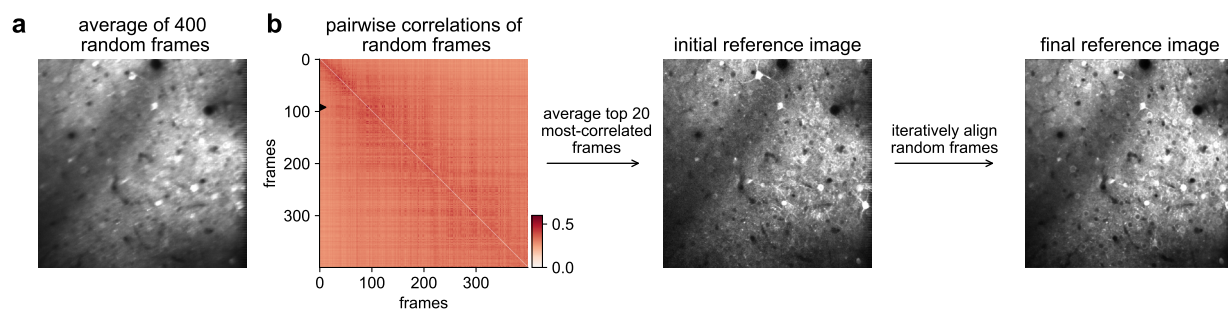
Motion-corrected movie + ROIs



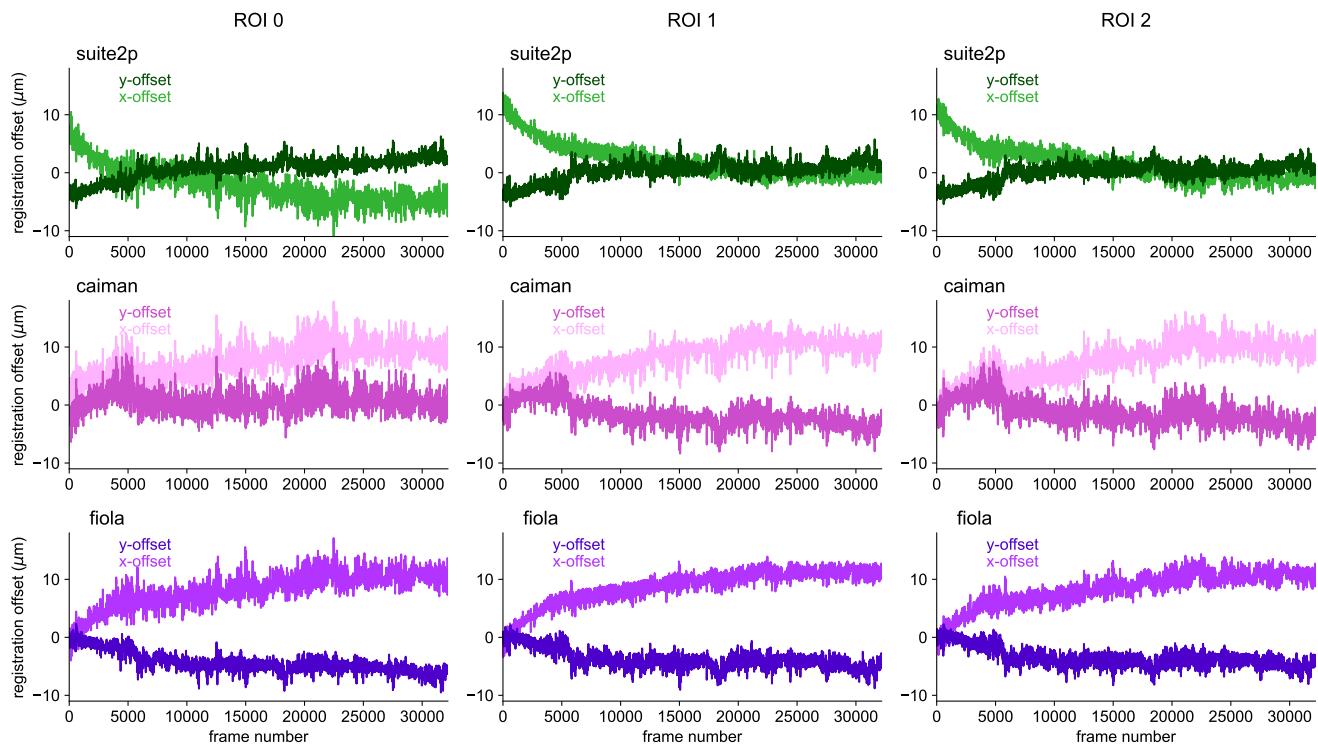
S1: Example functionality of the Suite2p graphical user interface.



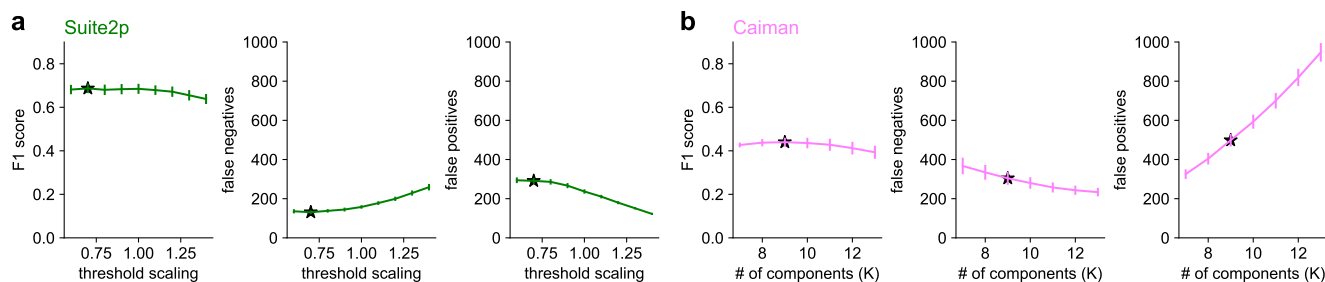
S2: All ROIs detected in 100,000+ neuron recording. Each panel shows the maximum projection image (maximum over time) with the detected somatic ROIs overlaid in random colors. Recording was made with a Thorlabs Bergamo2, Toptica Femtofiber Ultra 920 @ 30Mhz and Nikon 10x, 0.5, Plan Apo, Glyc with water immersion in a riboL1-gcamp8s Tigre x Slc17a7 mouse.



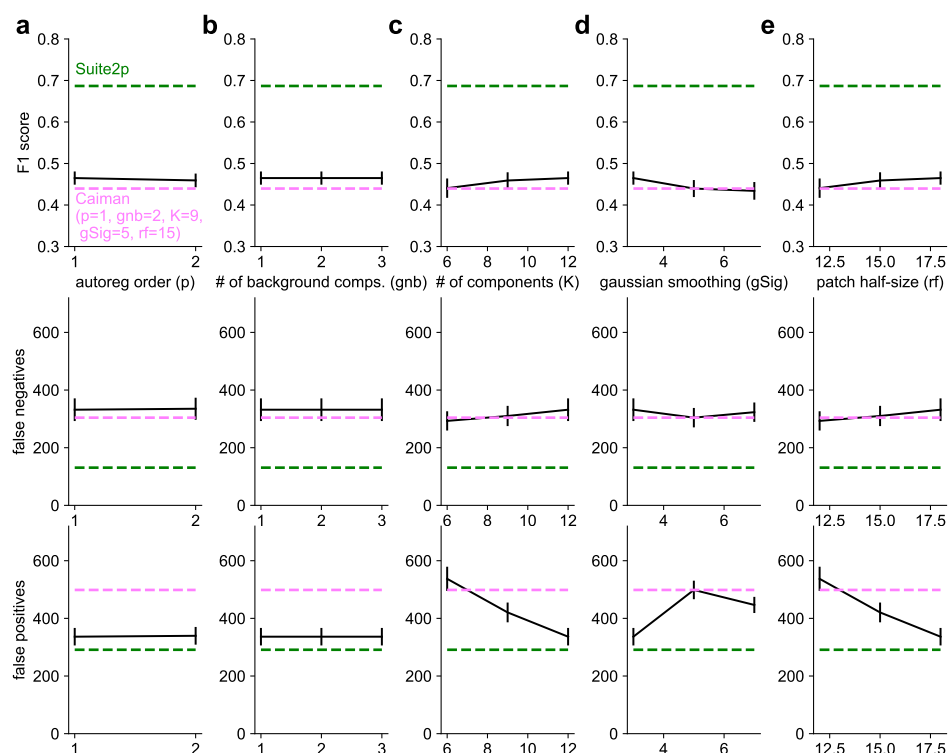
S3: Reference image computation. **a**, Average of 400 un-registered frames. **b**, Procedure to obtain the reference image by finding correlated frames and iteratively aligning them to an increasingly sharper reference image. Left: pairwise correlations of the random frames, with the frame row with the top 20 most-correlated pairs denoted by the black triangle. Middle: Average of 20 most-correlated frames. Right: Average of random frames after iterative alignment.



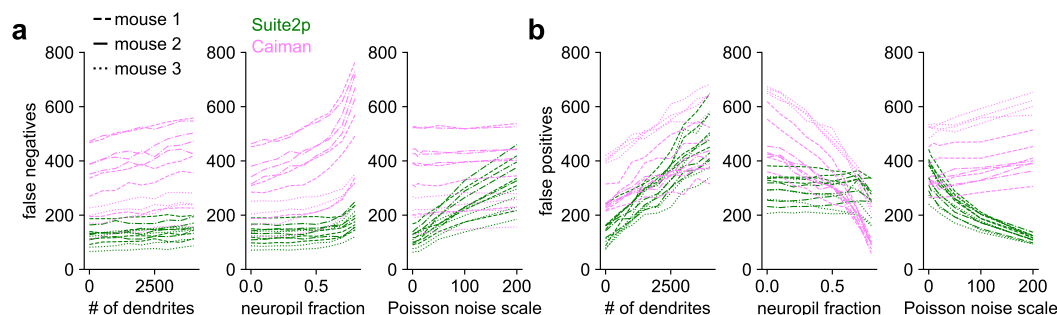
S4: Registration offsets from the experiment in Figure 3. Compared to Suite2p and Fiola, the registration offsets in Caiman span a shorter range, likely due to the continuously changing reference frame.



S5: Optimal cell detection thresholds a, Suite2p, b, Caiman.



S6: Parameter sweep for Caiman. Each panel shows the detection performance metric (F1 score) as a function of a parameter, after taking the maximum across all other parameters. While this procedure in general overfits, the improvements are minimal regardless. The parameters swept over are: **a**, Autoregressive model order. **b**, Number of components for modelling the background / neuropil per patch. **c**, Number of components per patch. **d**, Gaussian smoothing parameter. **e**, Patch half-size in pixels.



S7: Additional metrics when varying simulation parameters. a False negatives. **b**, False positives.